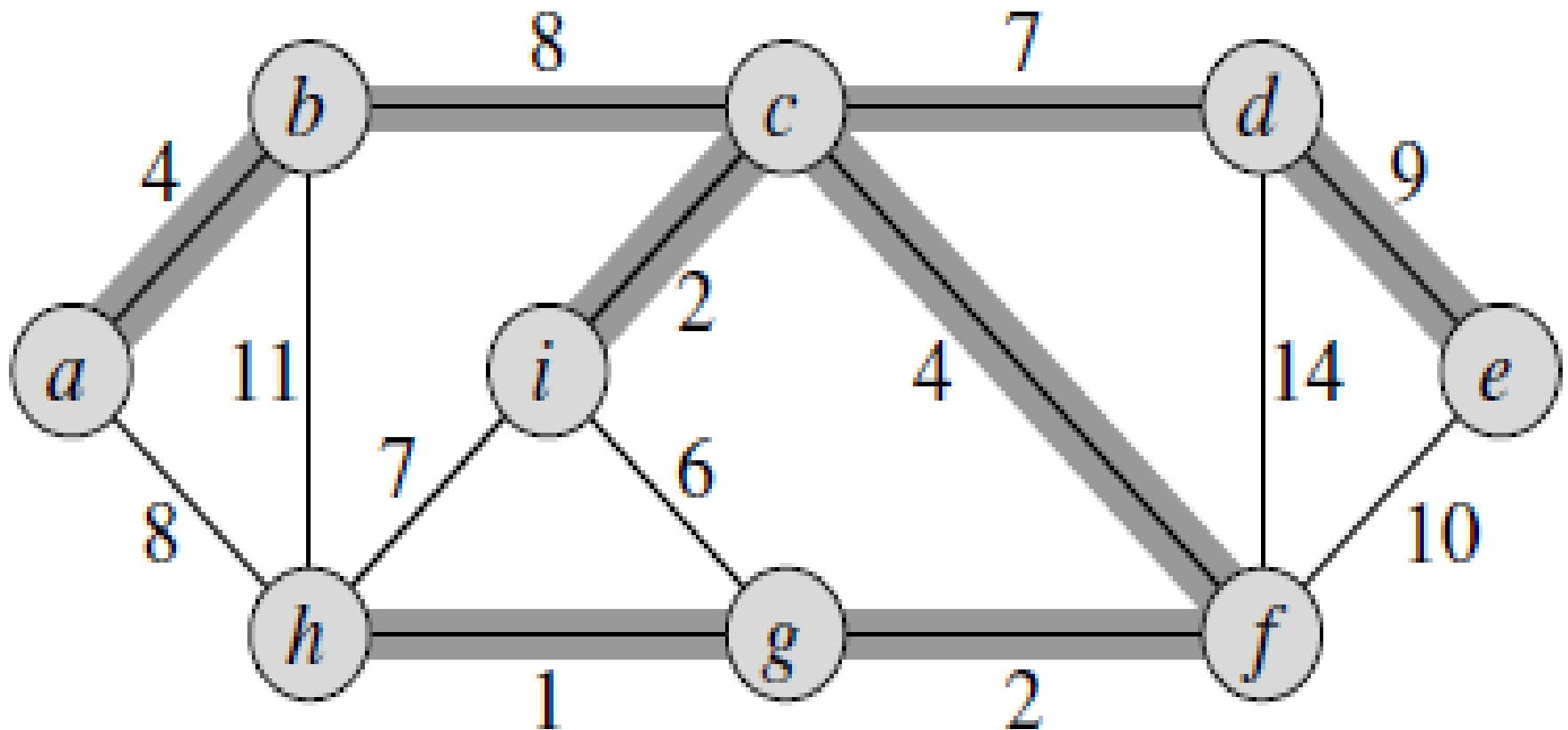


Algoritmo de Kruskal

Problema: Dado um grafo conexo G com pesos positivos nas arestas, determinar uma **árvore geradora** de G com custo mínimo.



Algoritmo de Kruskal

O Algoritmo de Kruskal utiliza uma técnica de programação conhecida como **Método Guloso**, cuja estratégia geral é: **“a cada nova iteração, acrescente à solução parcial a parte mais apetitosa”**.

Algoritmo de Kruskal

Dado o grafo G , o algoritmo constrói uma árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset$; -- *inicialização de T*

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$, $j = 1, \dots, m-1$

para $j = 1, \dots, m$ faça

se e_j não forma ciclo com as arestas em $E(T)$

então acrescente e_j a $E(T)$

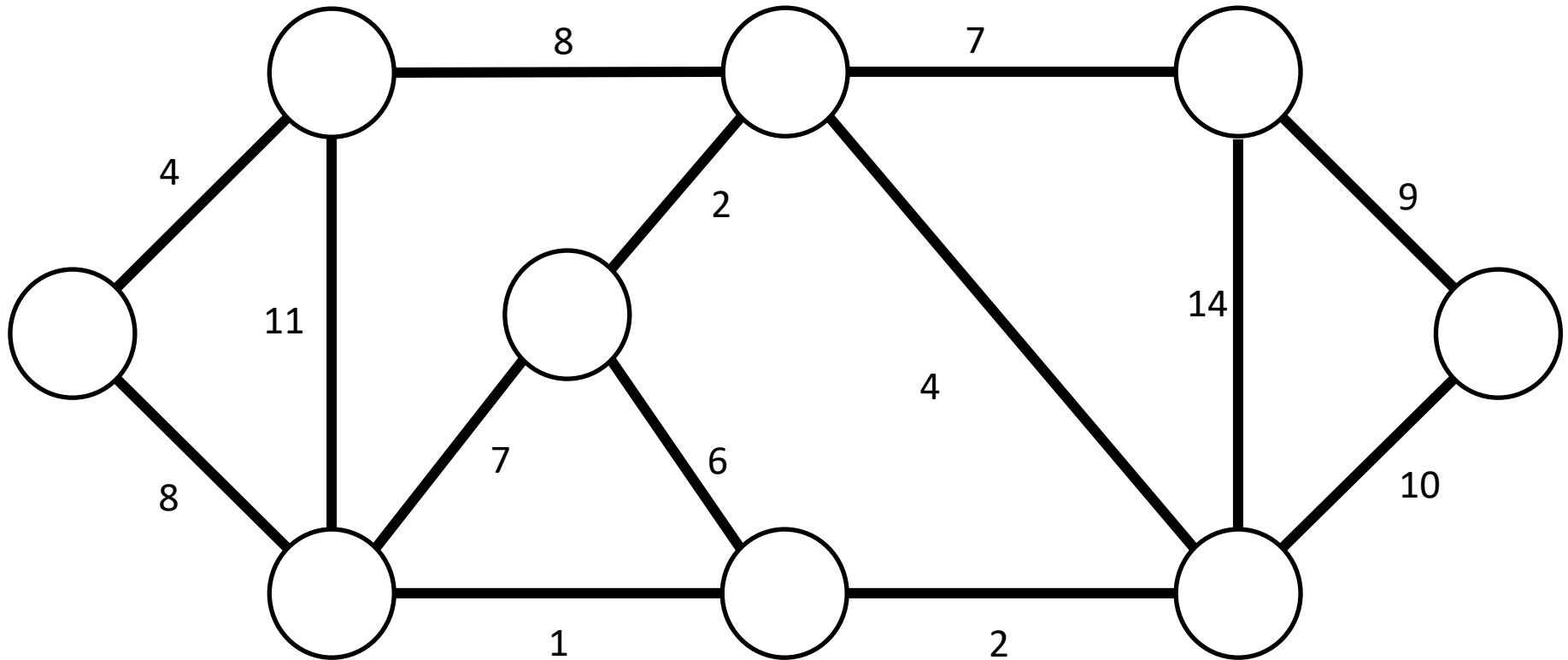
fim-se

fim-para

retorne T

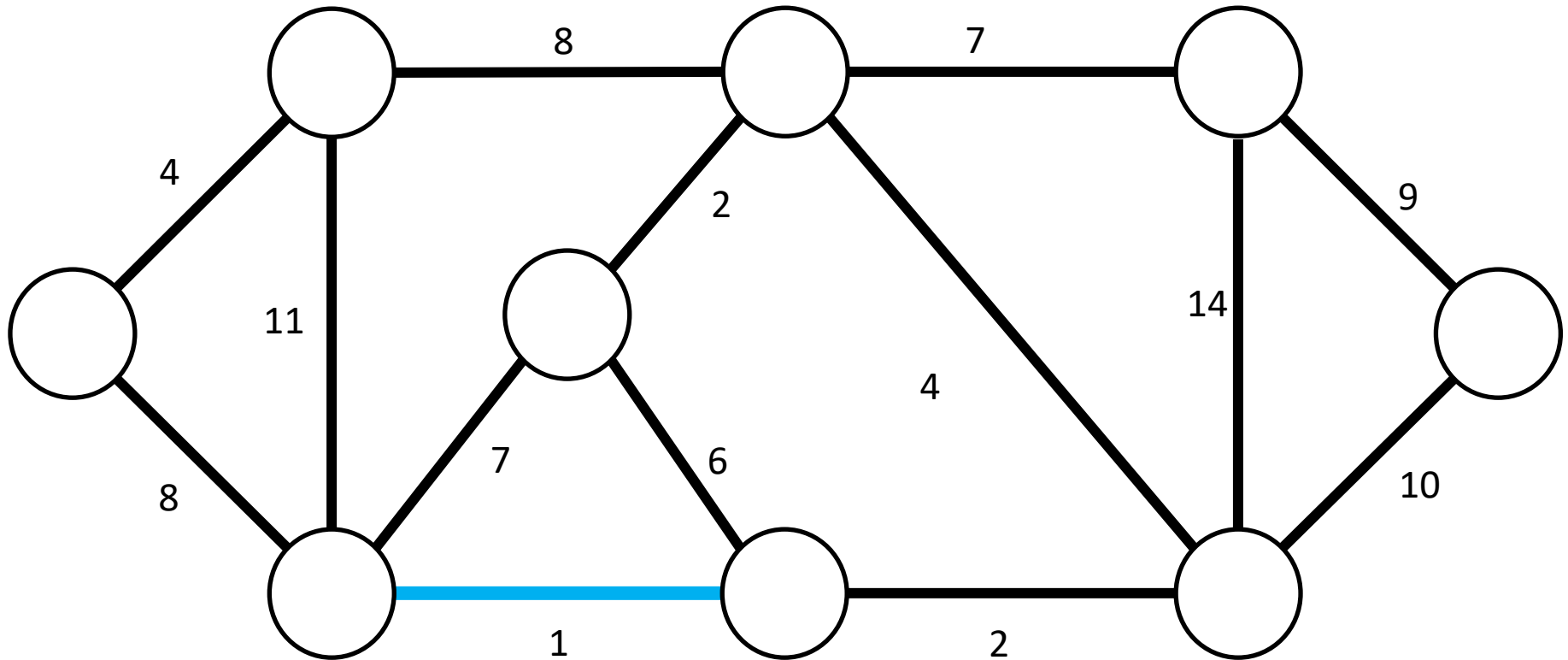
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



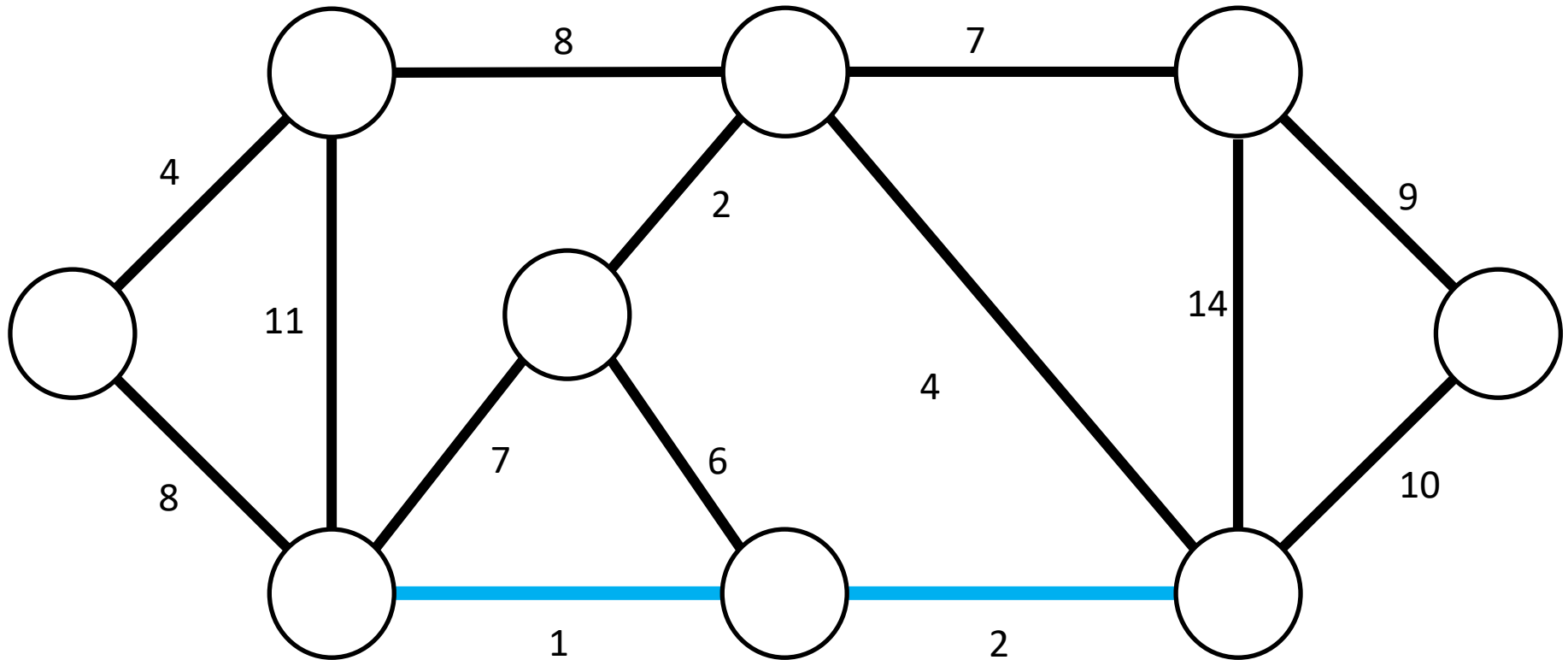
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



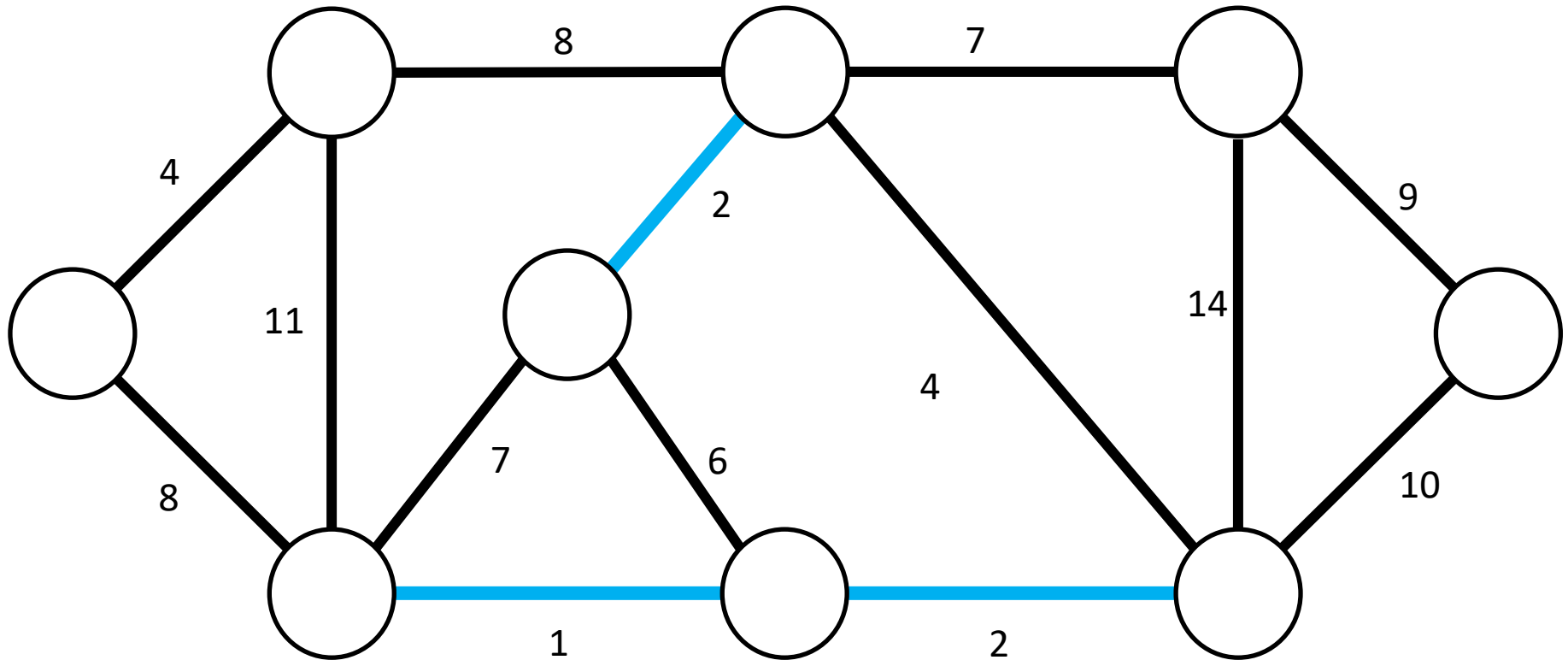
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



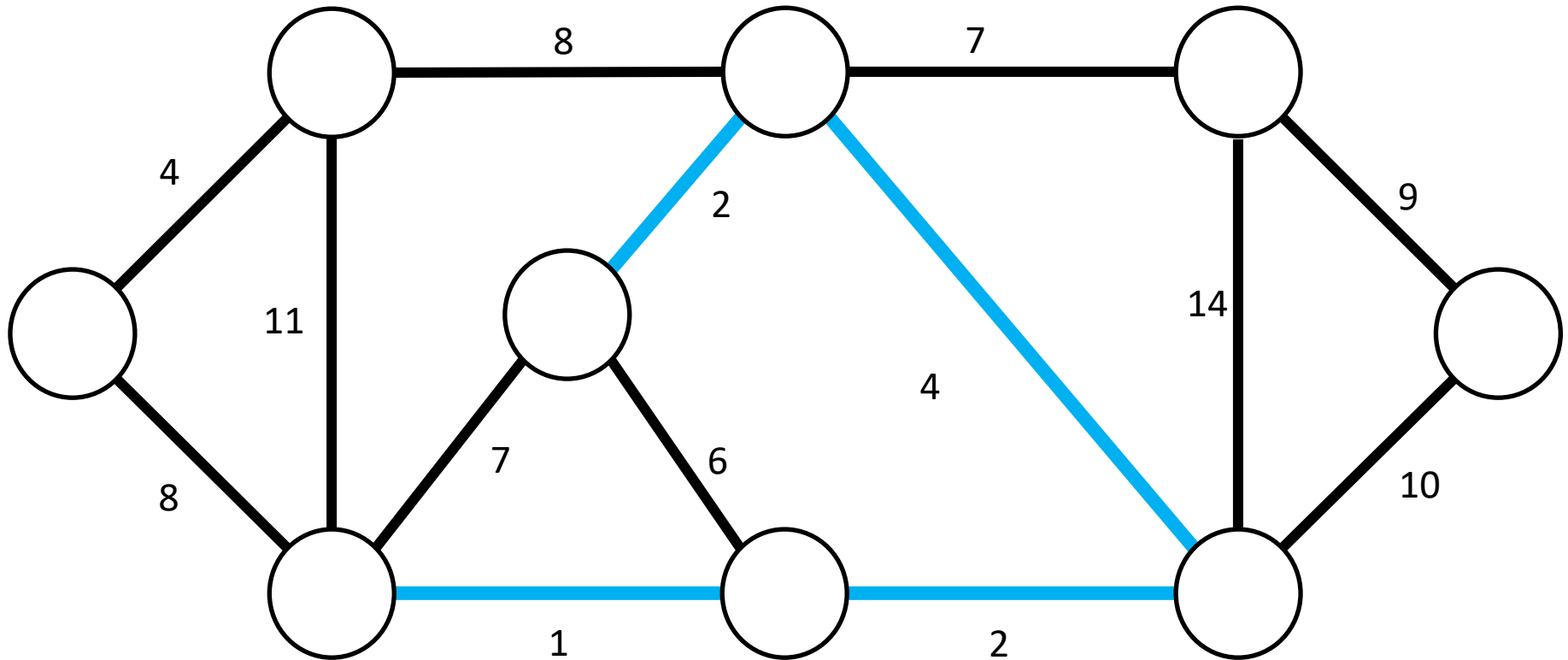
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



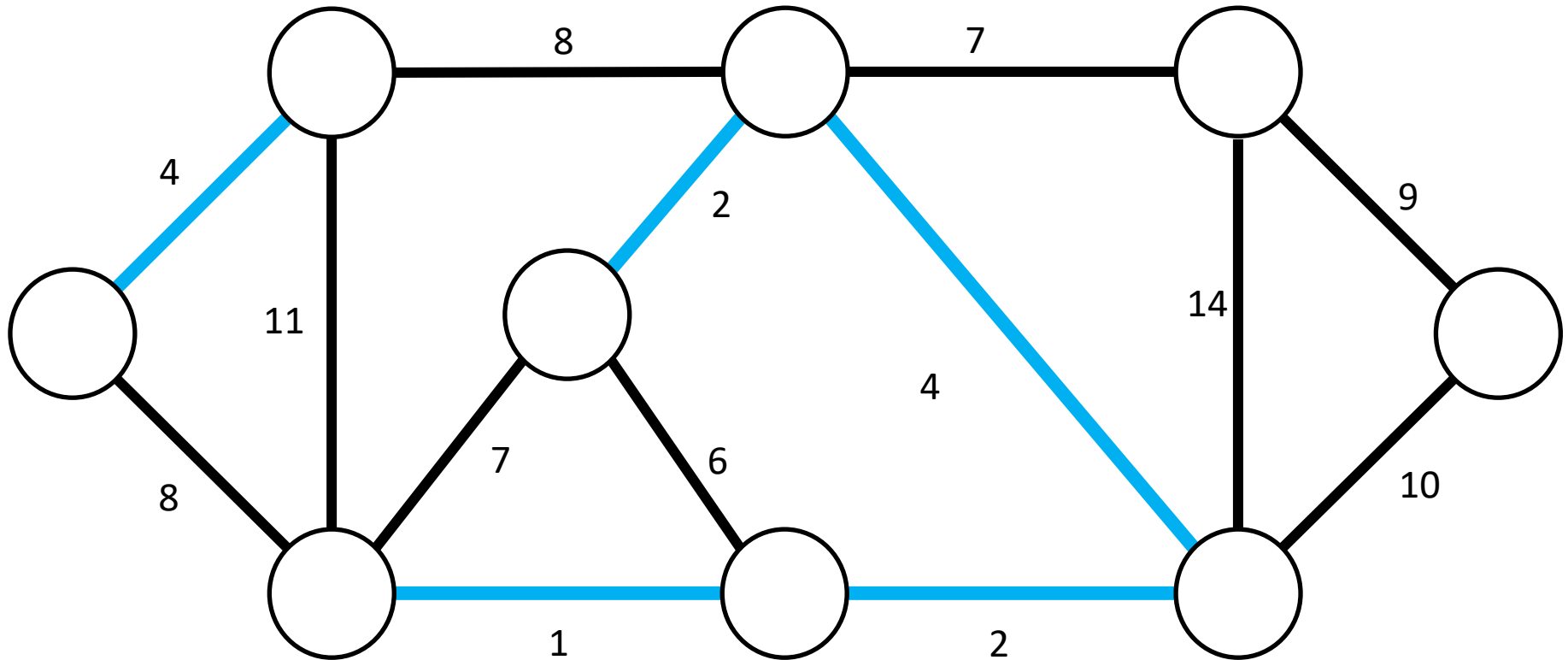
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



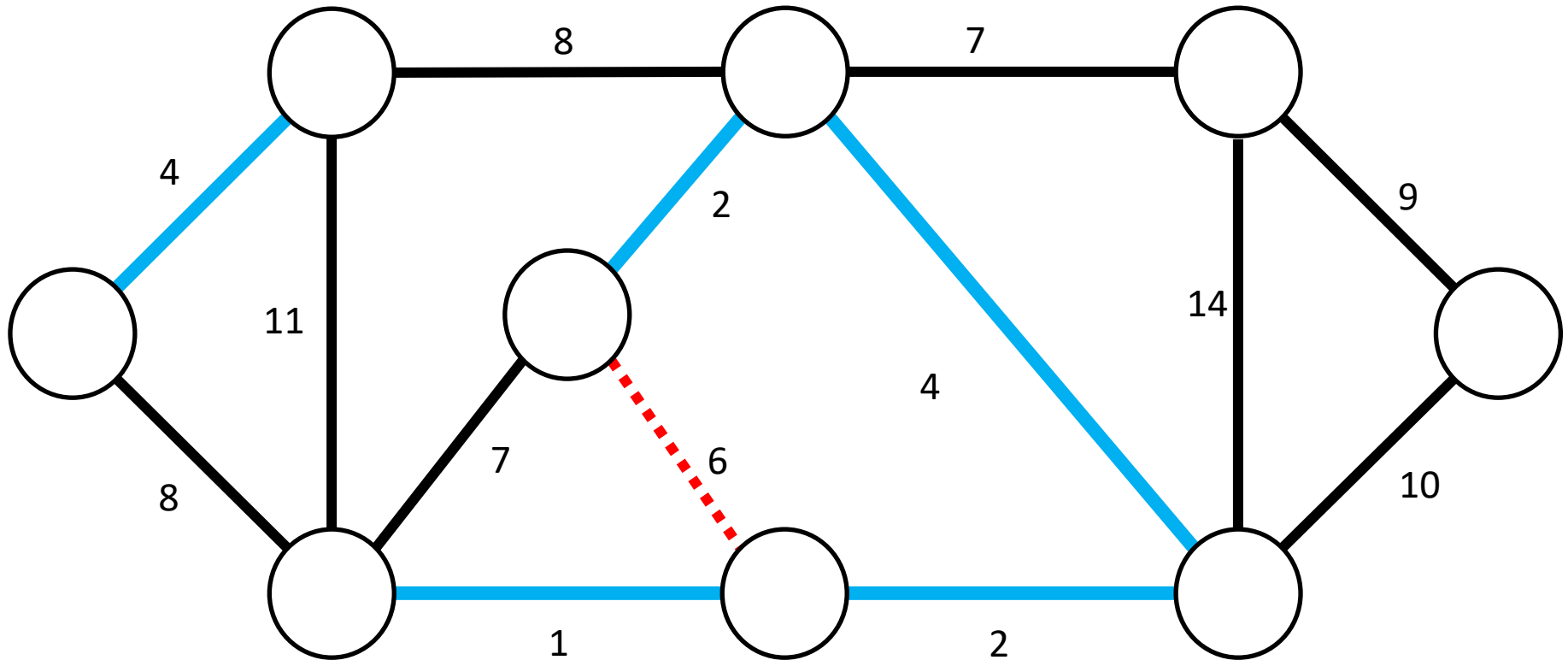
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



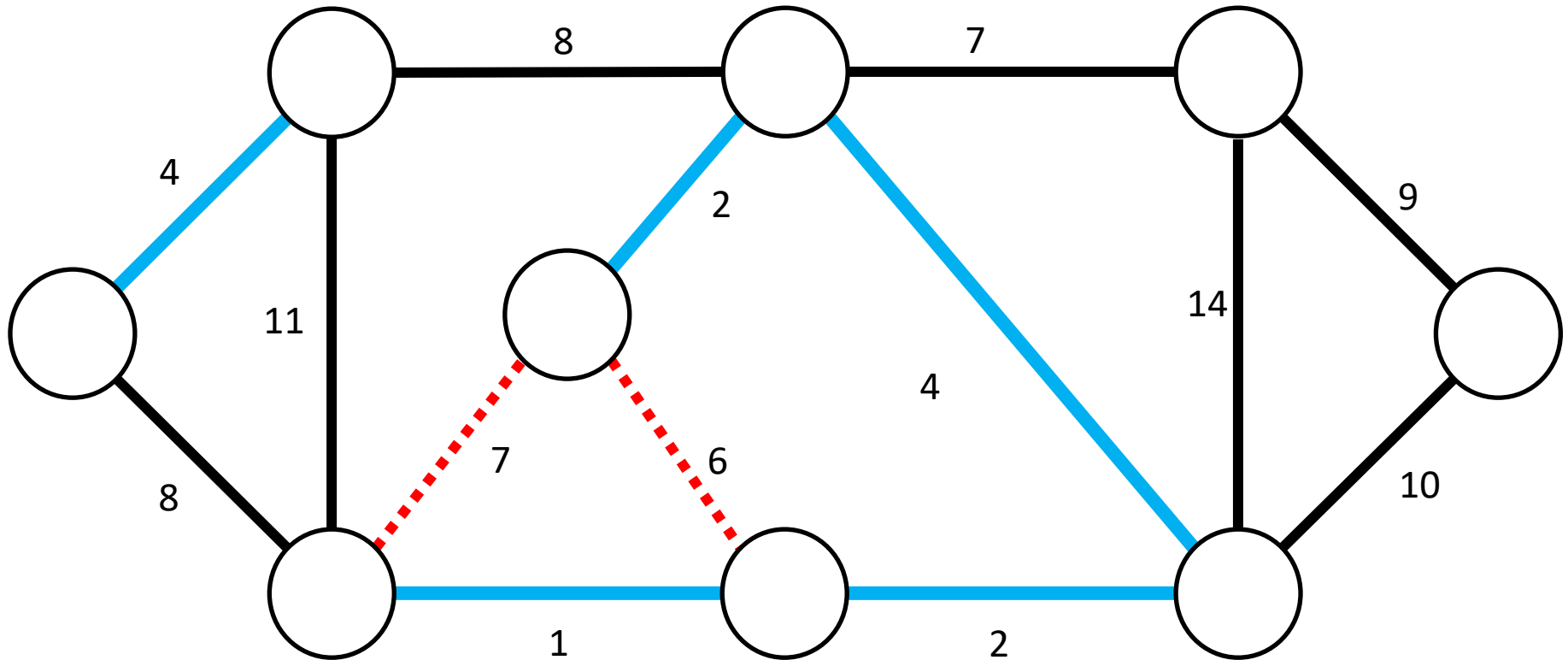
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



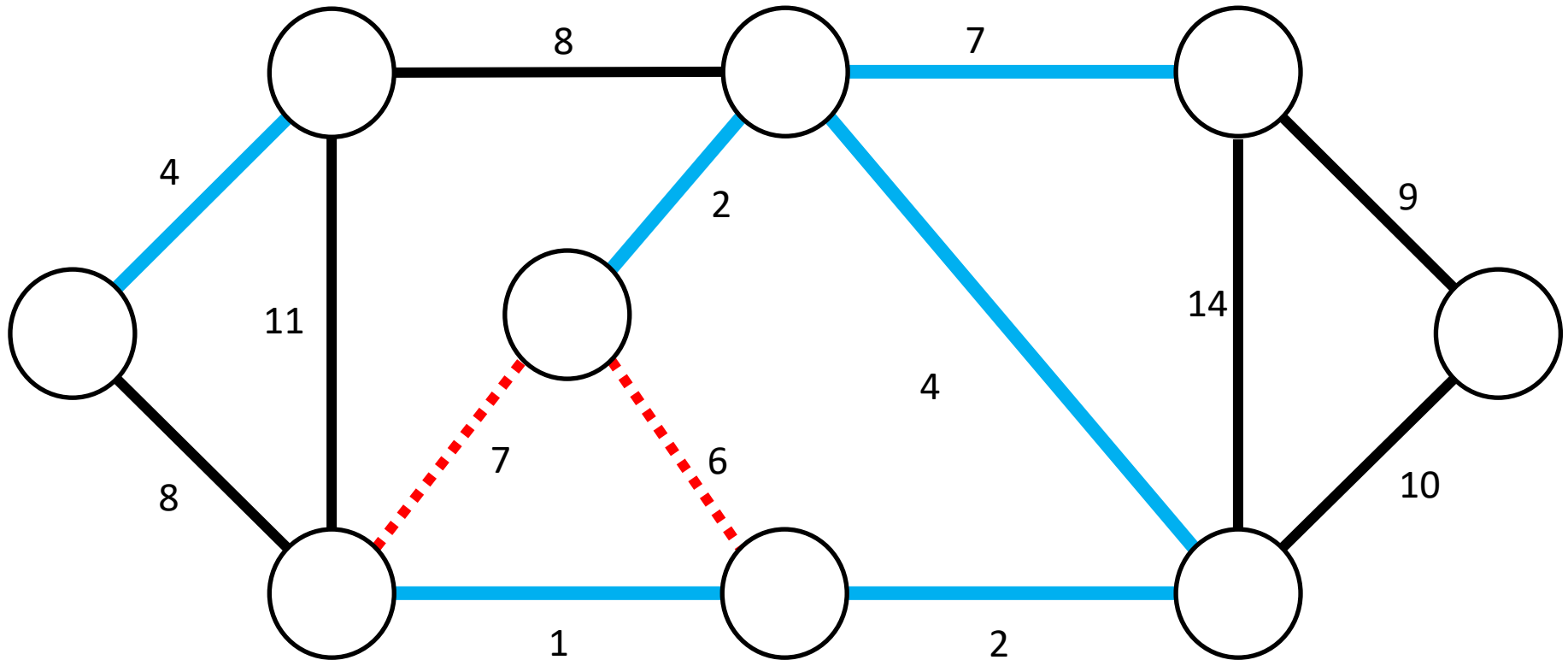
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



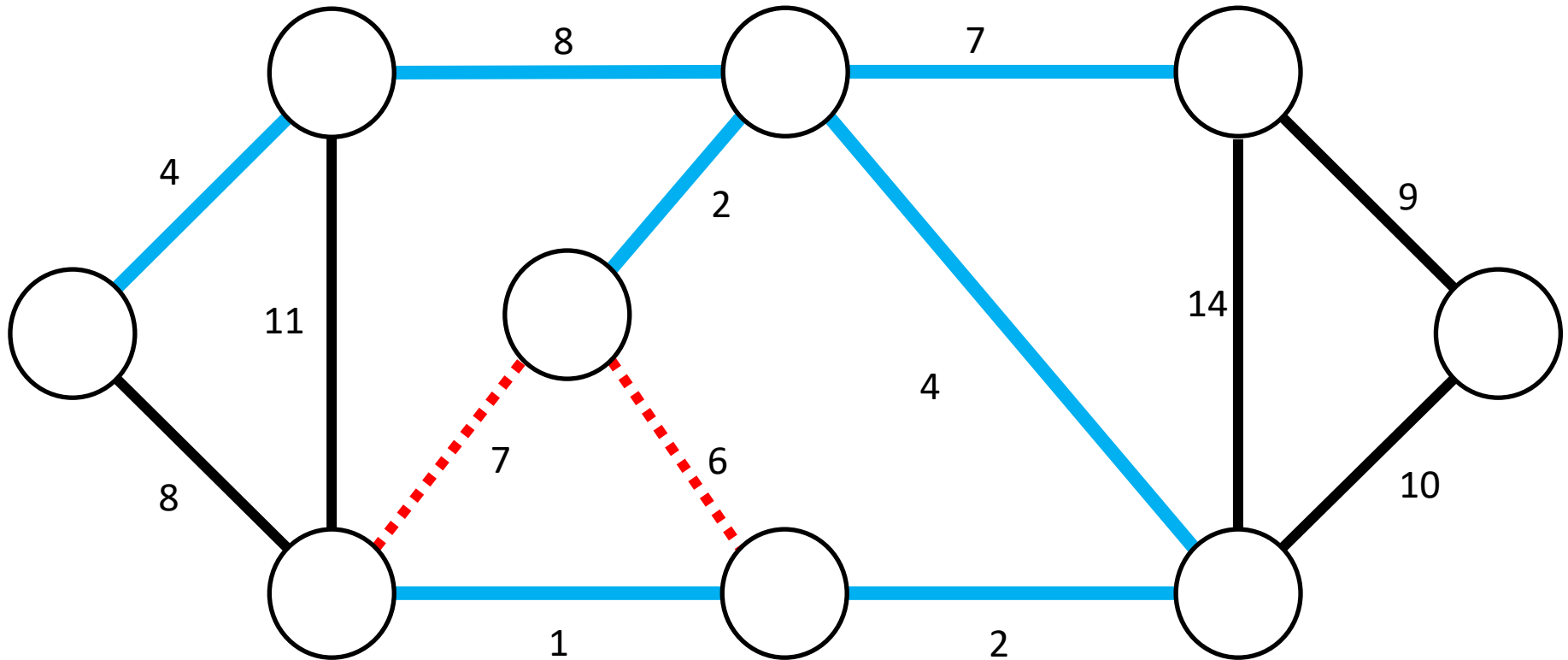
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



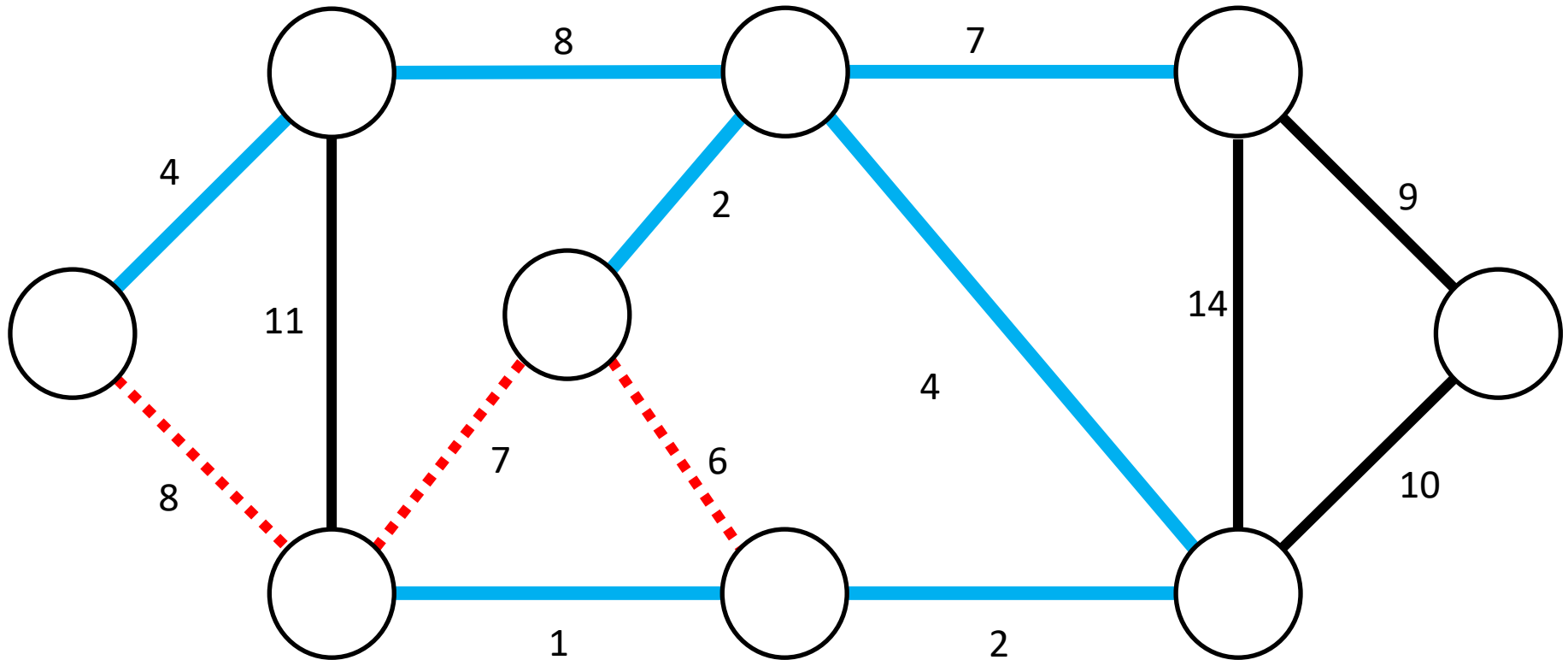
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



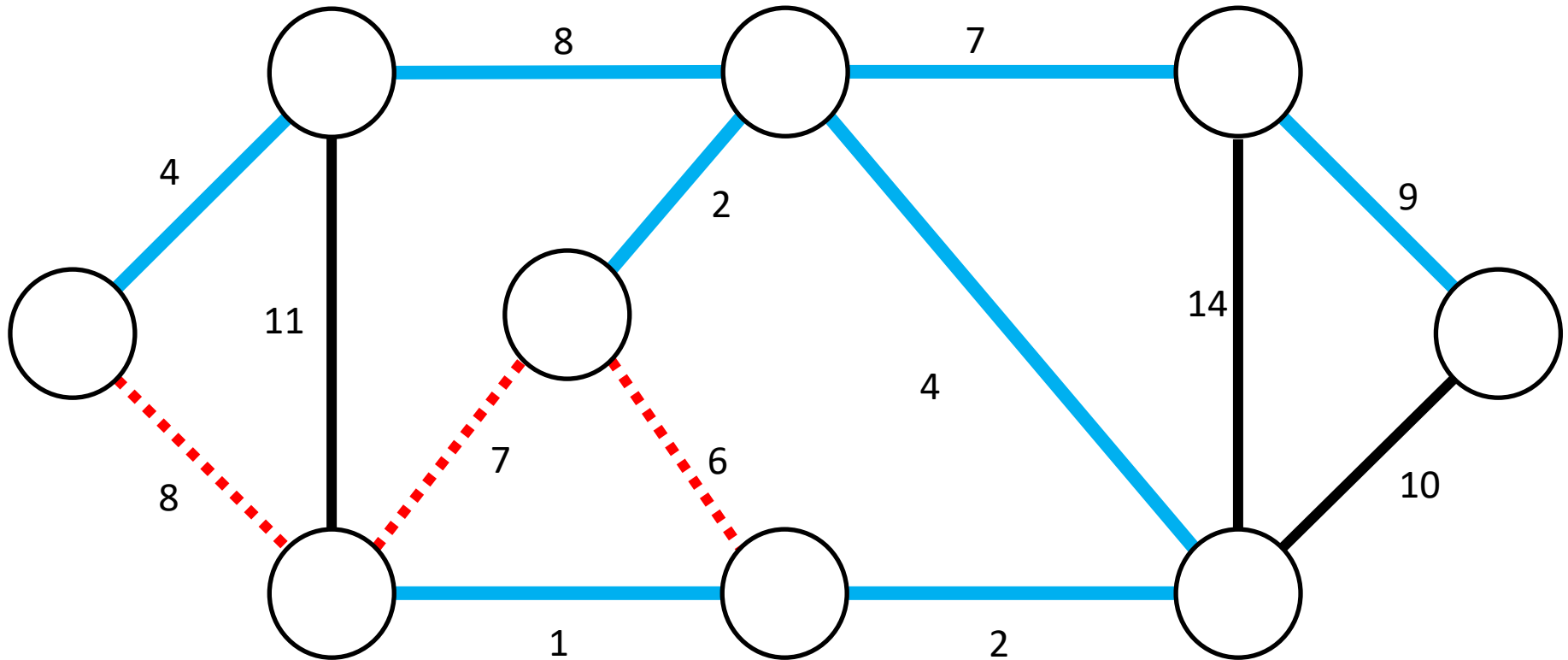
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



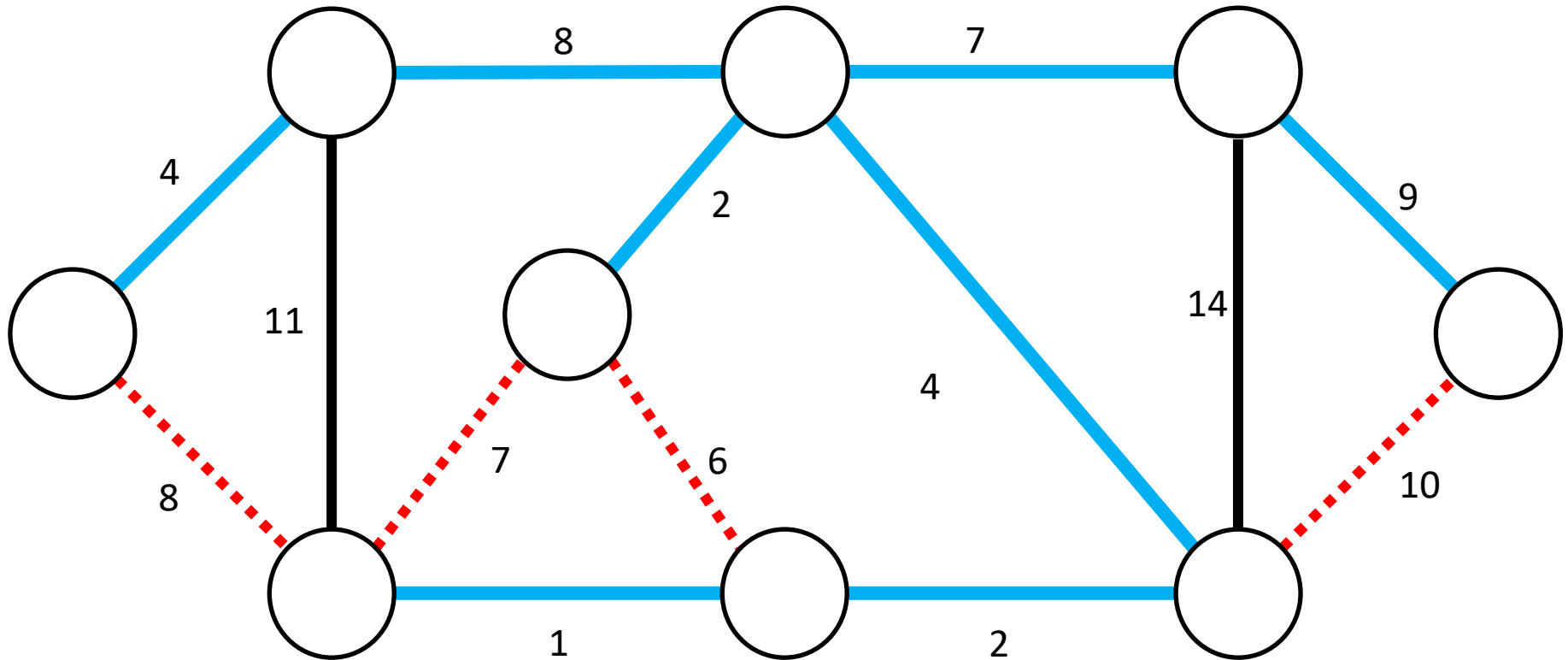
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



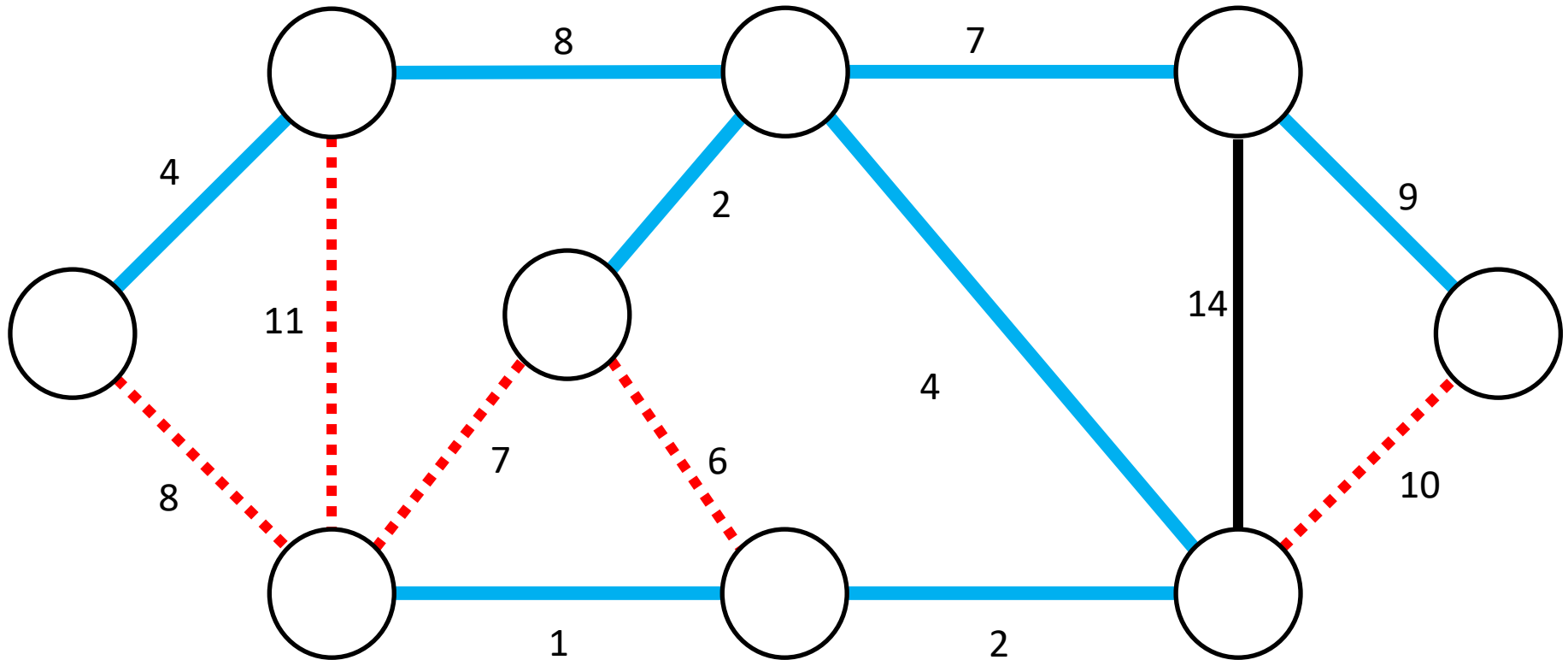
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



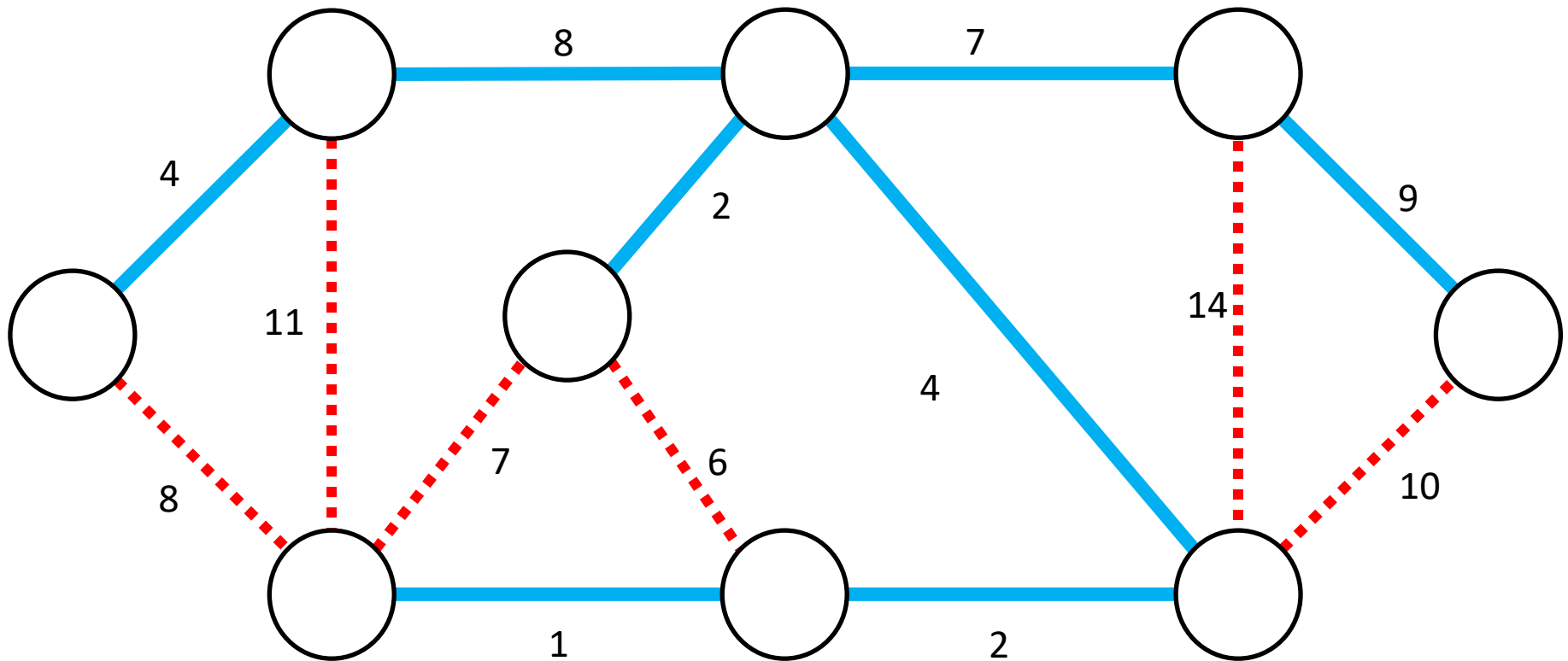
Algoritmo de Kruskal

Exemplo: Simulação da execução do algoritmo.



Algoritmo de Kruskal

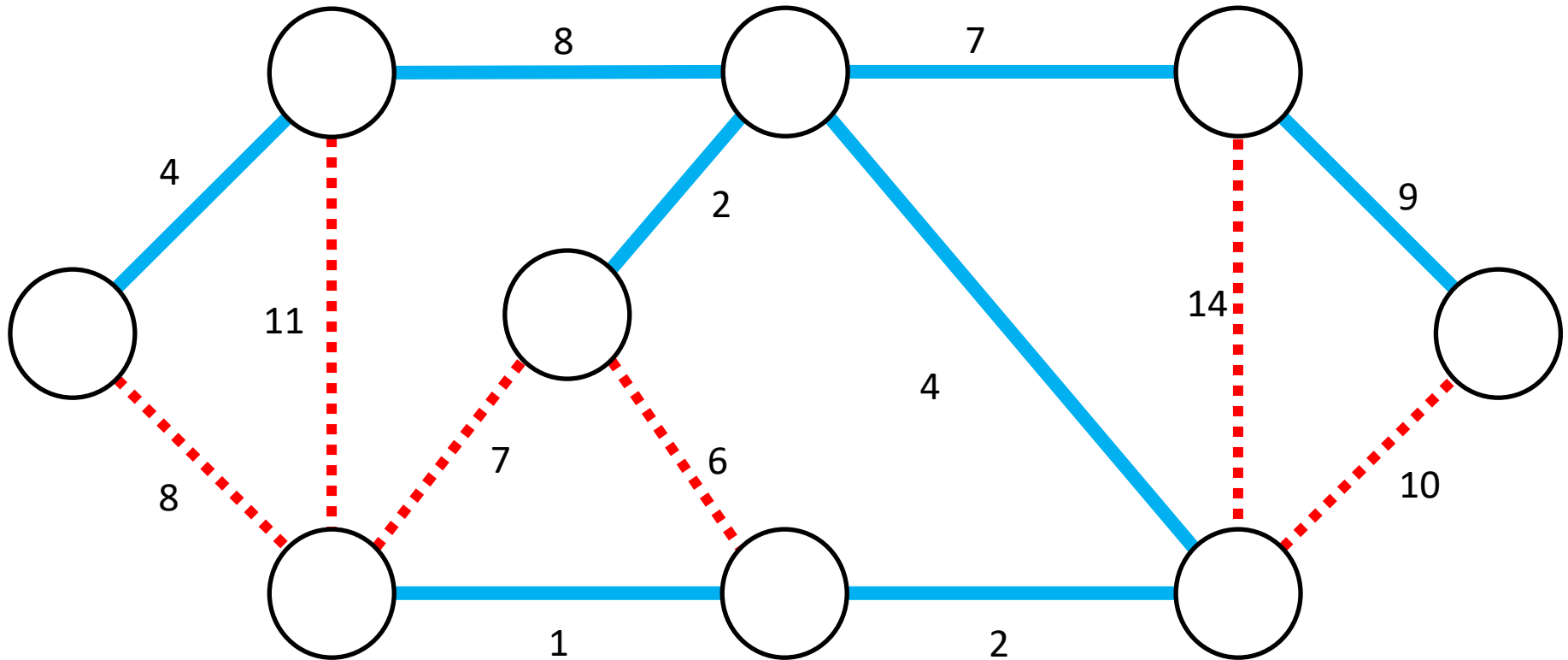
Exemplo: Simulação da execução do algoritmo.



Algoritmo de Kruskal

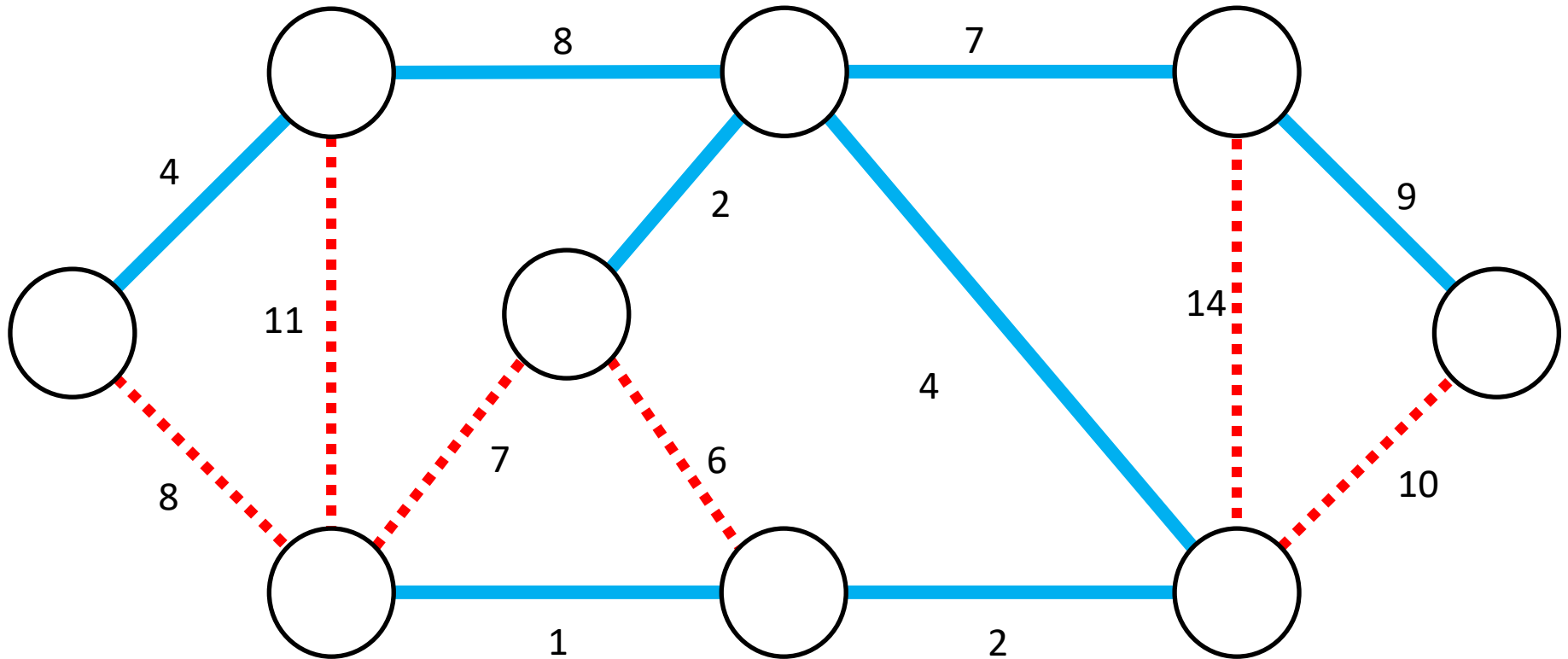
Exemplo: Simulação da execução do algoritmo.

custo final da árvore geradora: 37



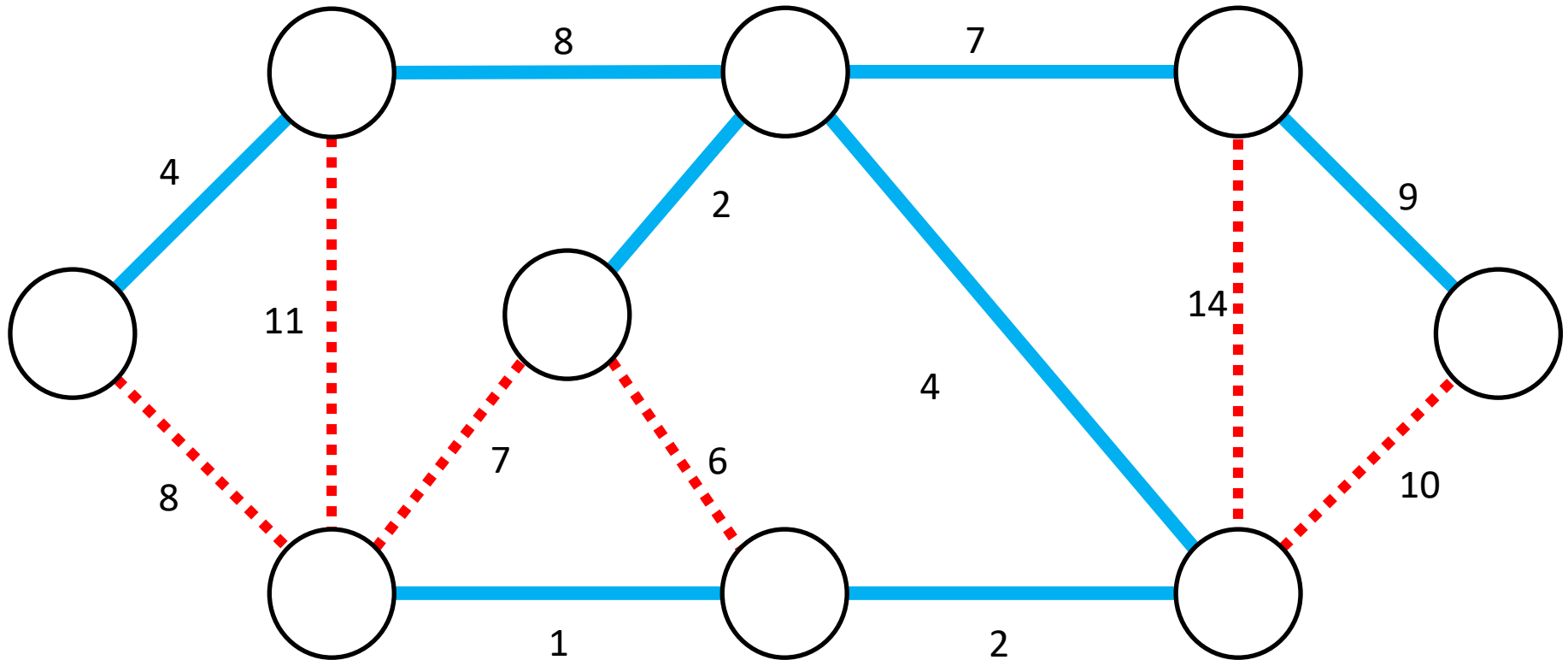
Algoritmo de Kruskal

Questão: Como saber de modo eficiente se a aresta em consideração forma ciclo com as azuis já escolhidas?



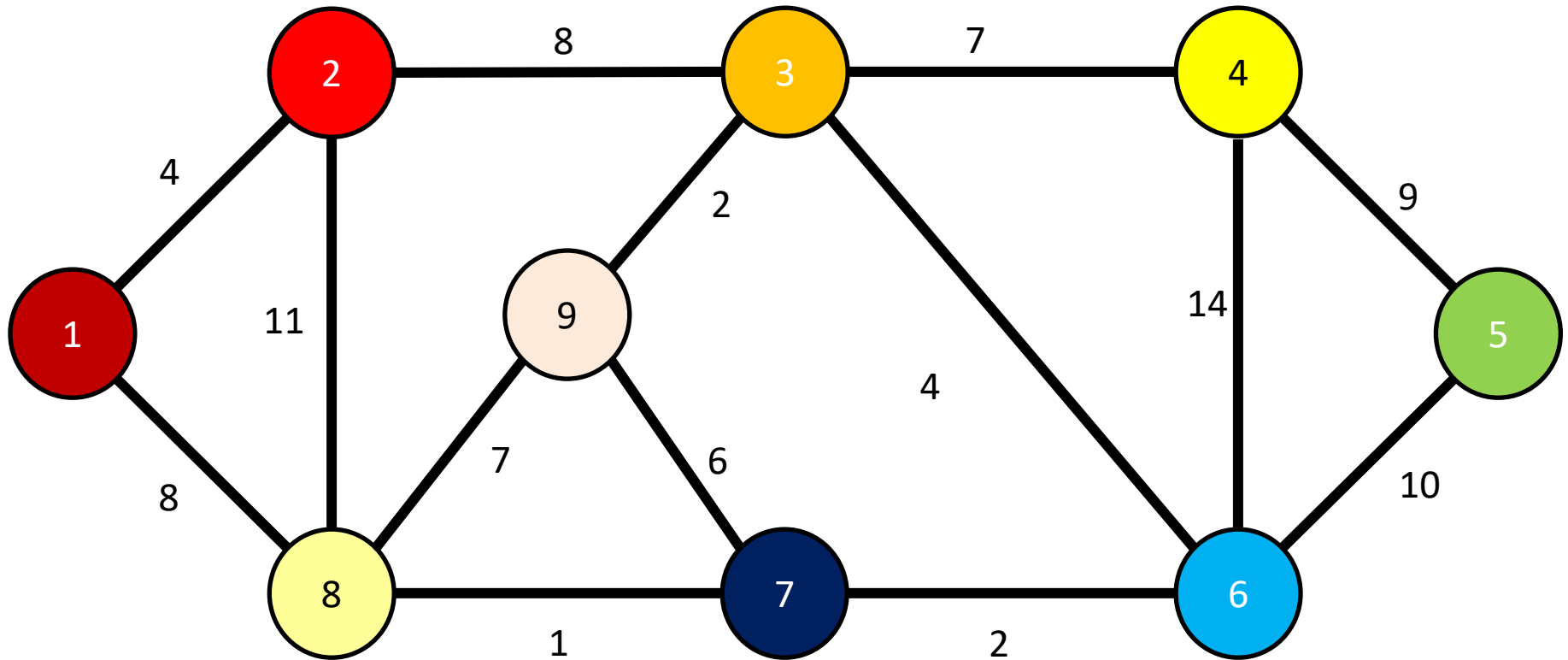
Algoritmo de Kruskal

Estratégia: Uma nova aresta pode ser incorporada à solução (floresta) parcial se ela une duas árvores distintas!



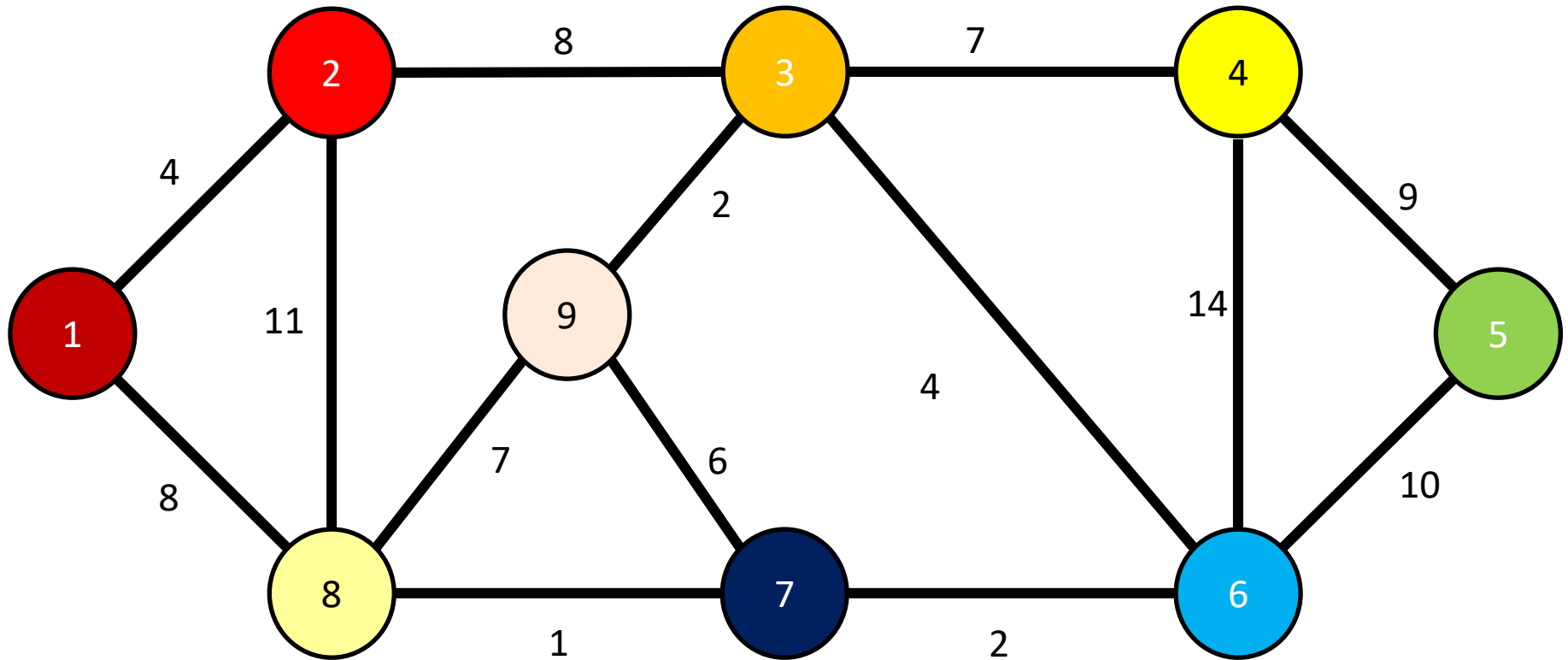
Algoritmo de Kruskal

Início: Os n vértices formam n árvores triviais distintas (cada uma com sua cor). Nenhuma aresta foi escolhida.



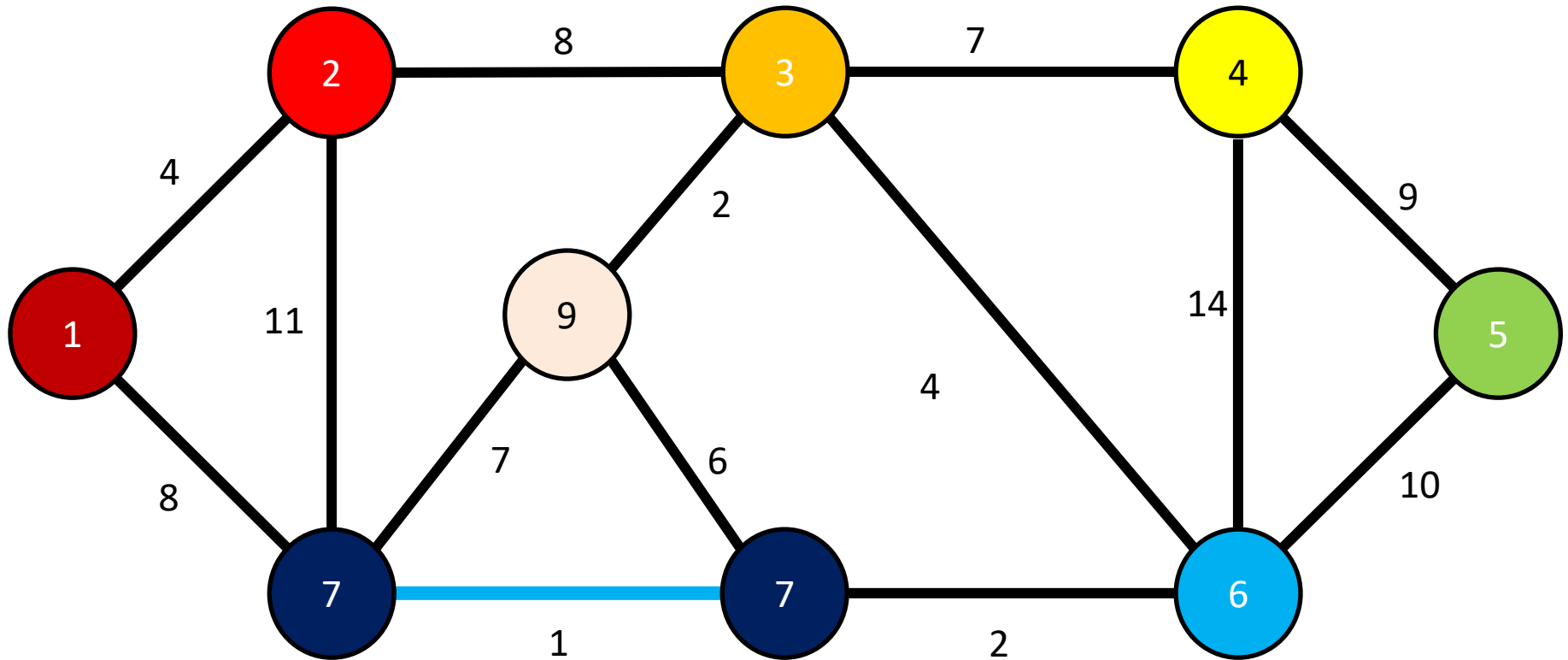
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



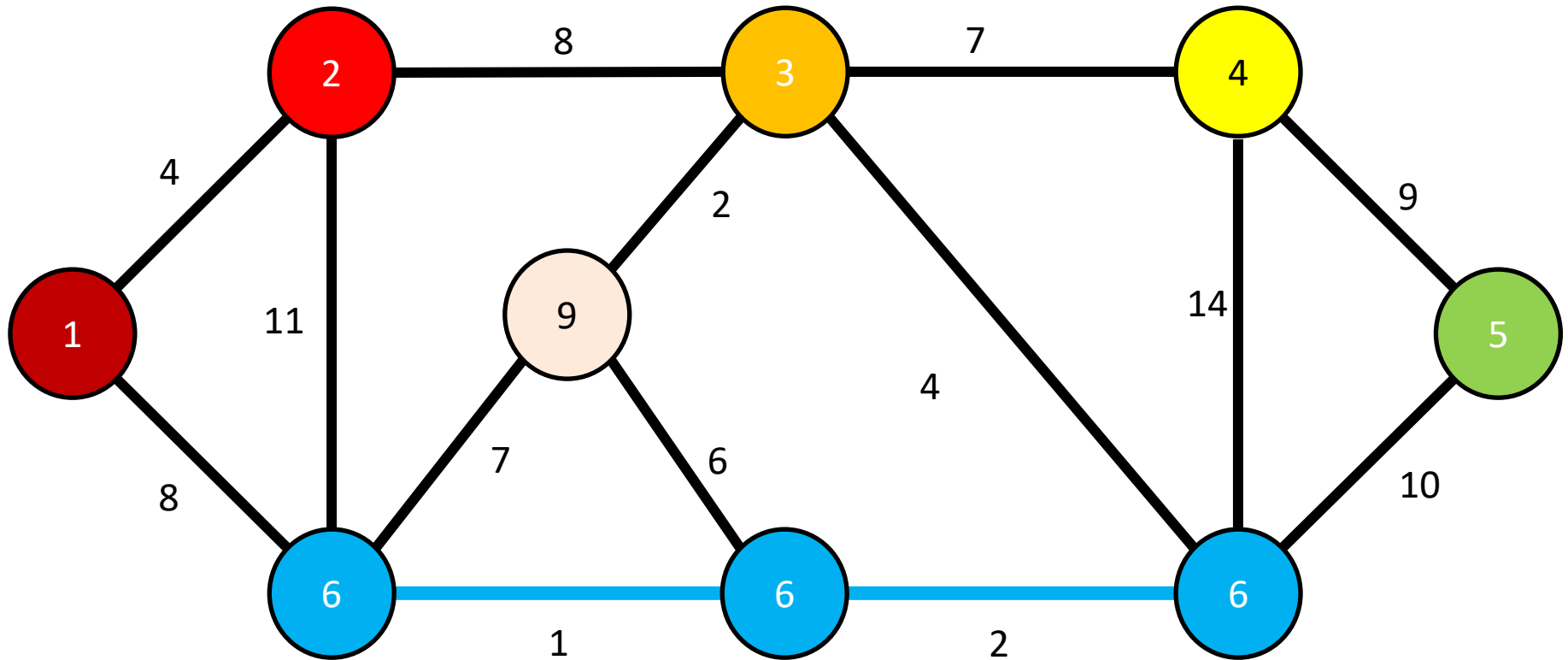
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



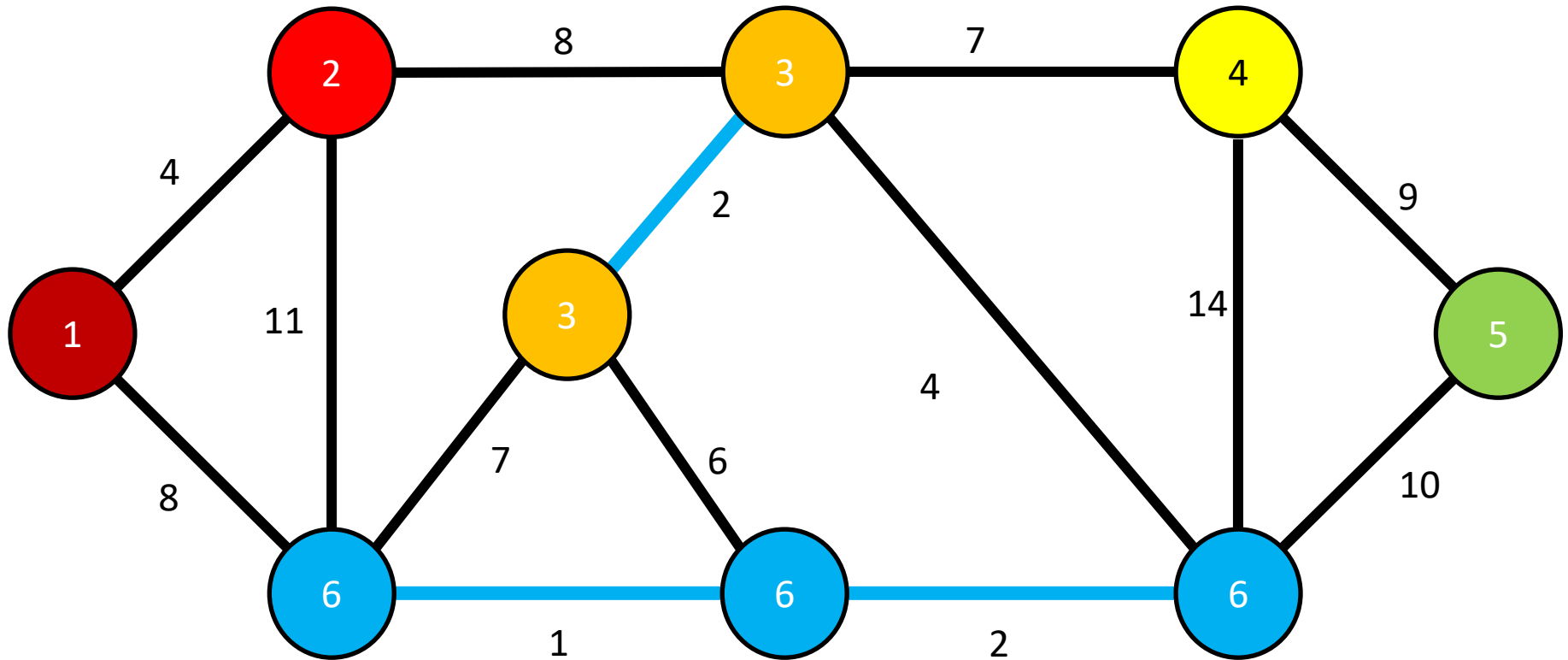
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



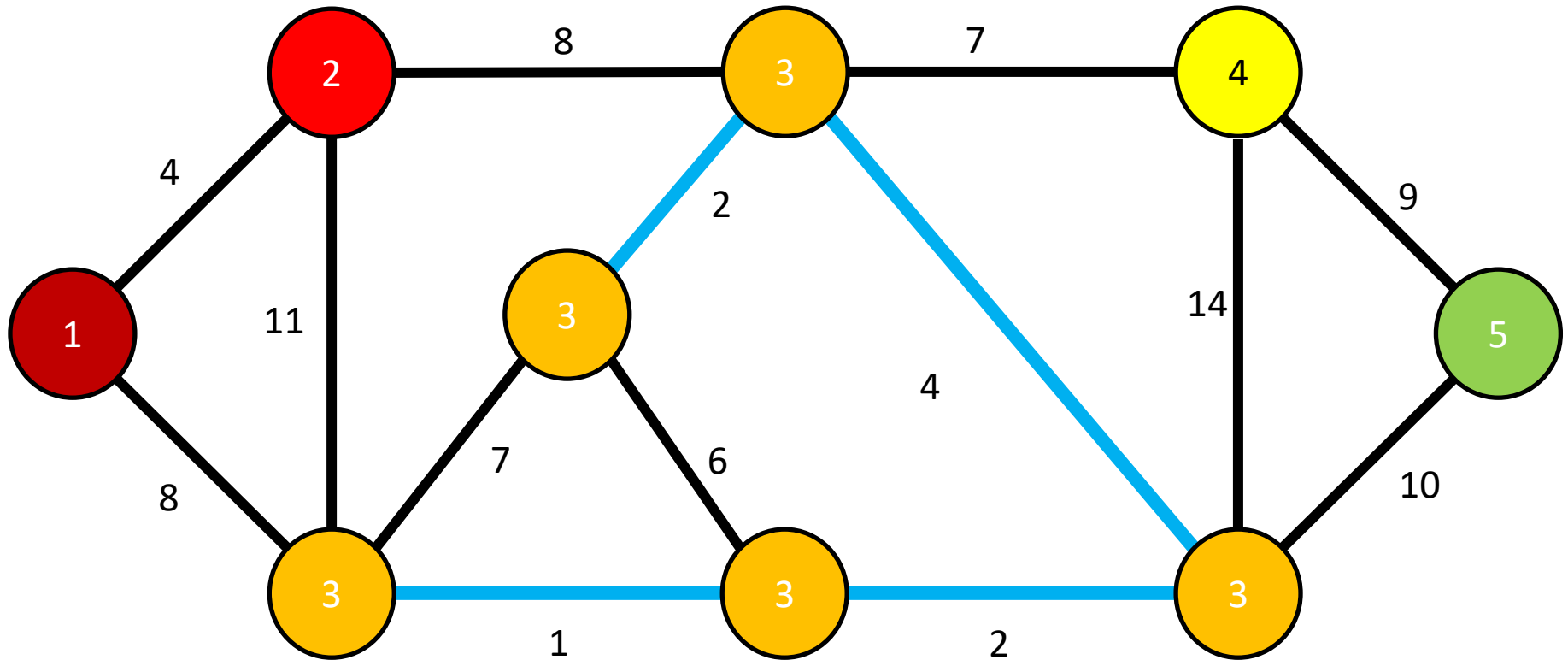
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



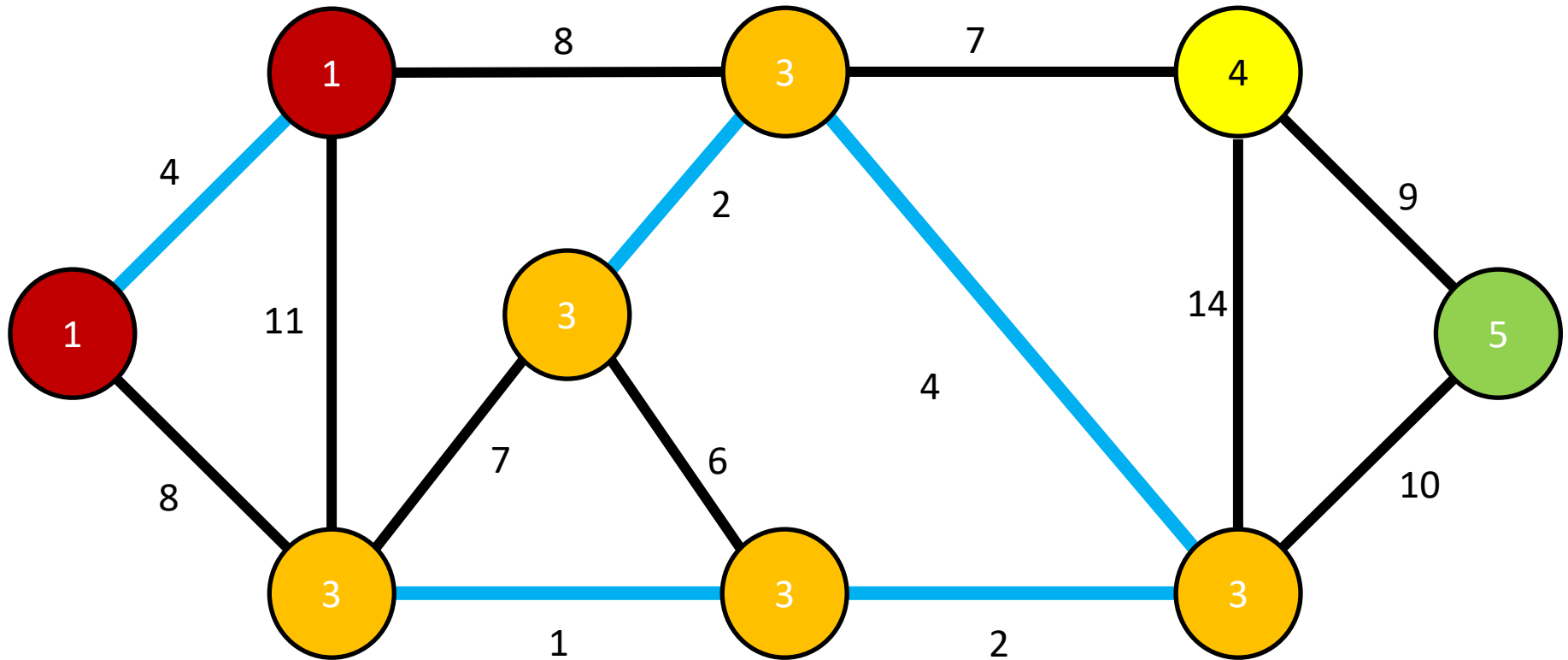
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



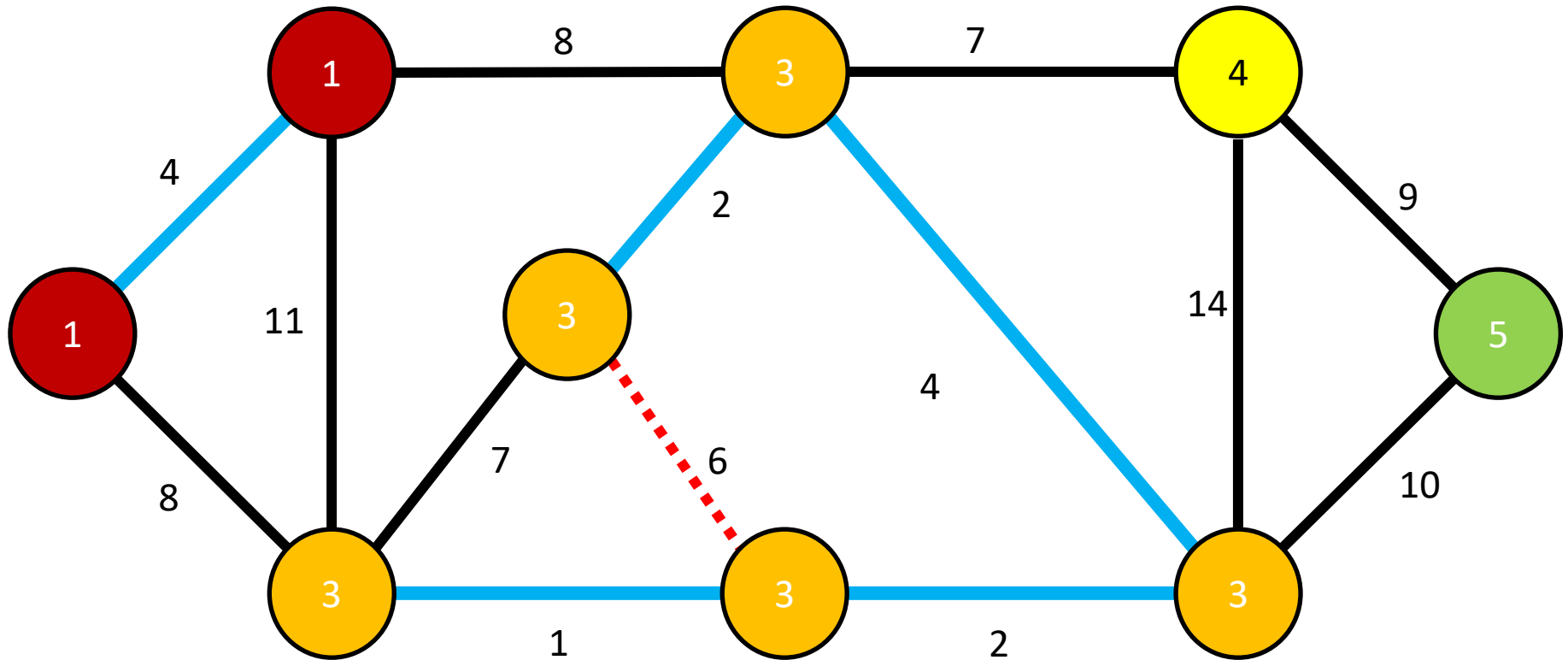
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



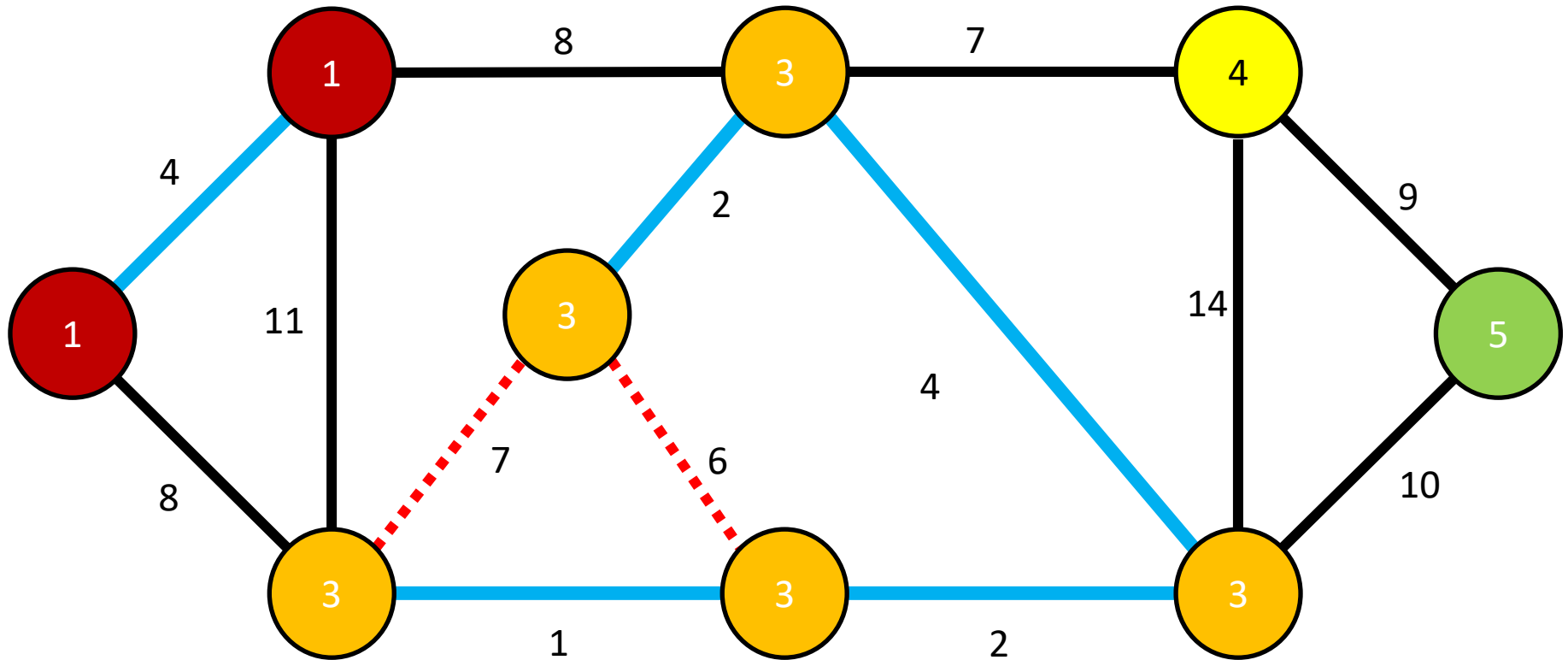
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



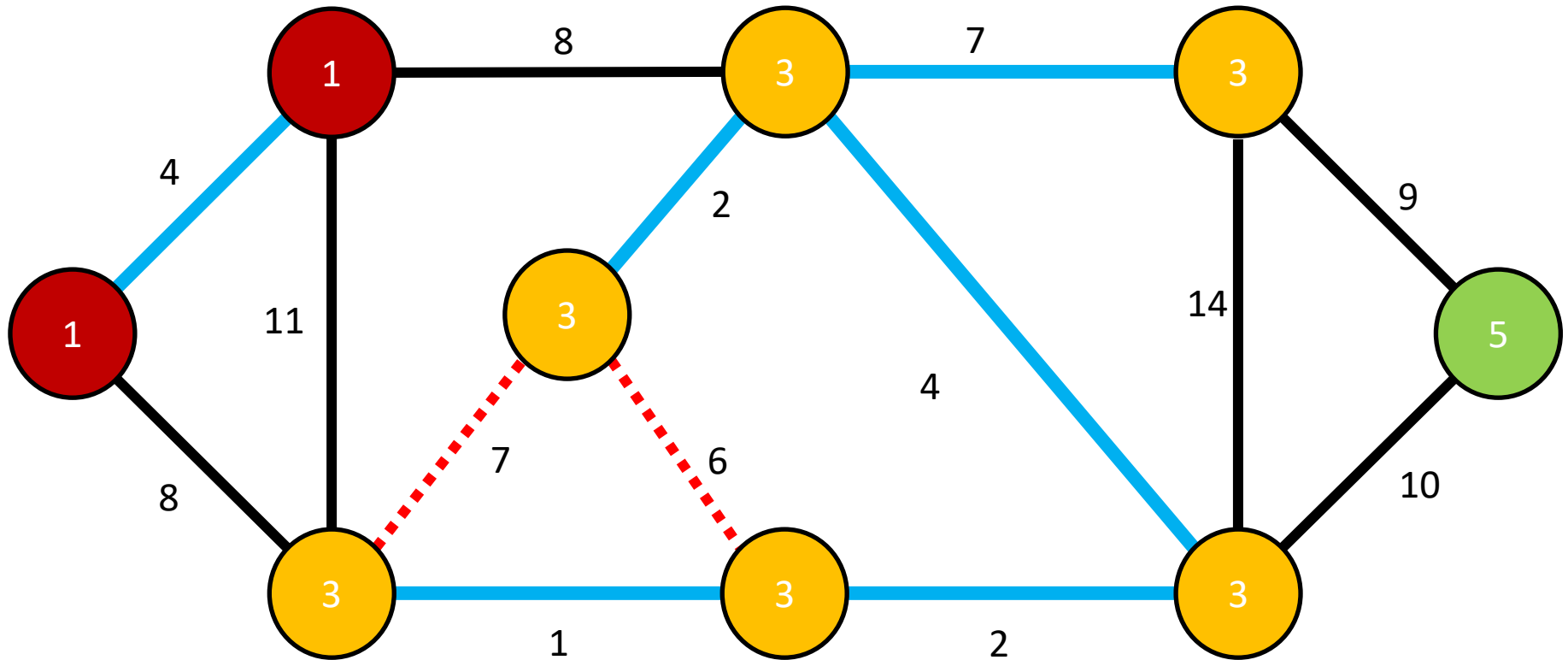
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



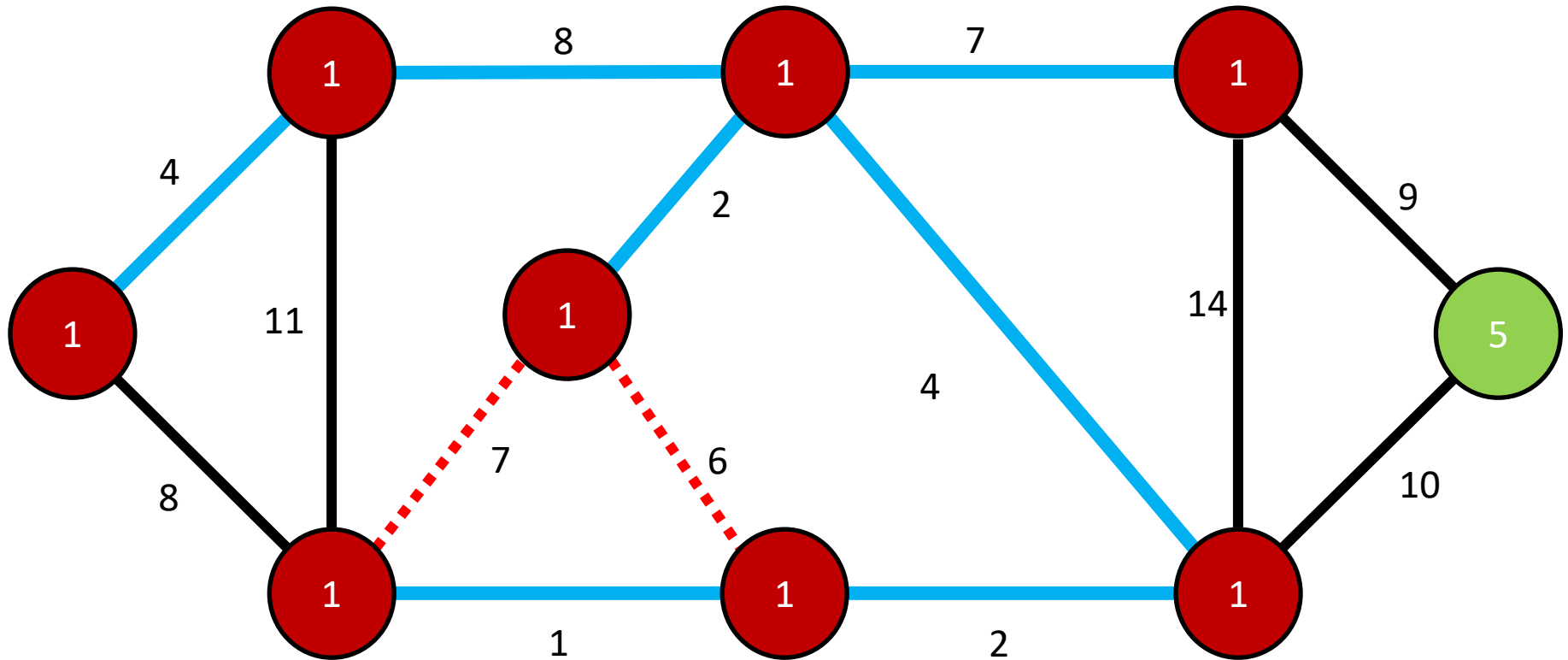
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



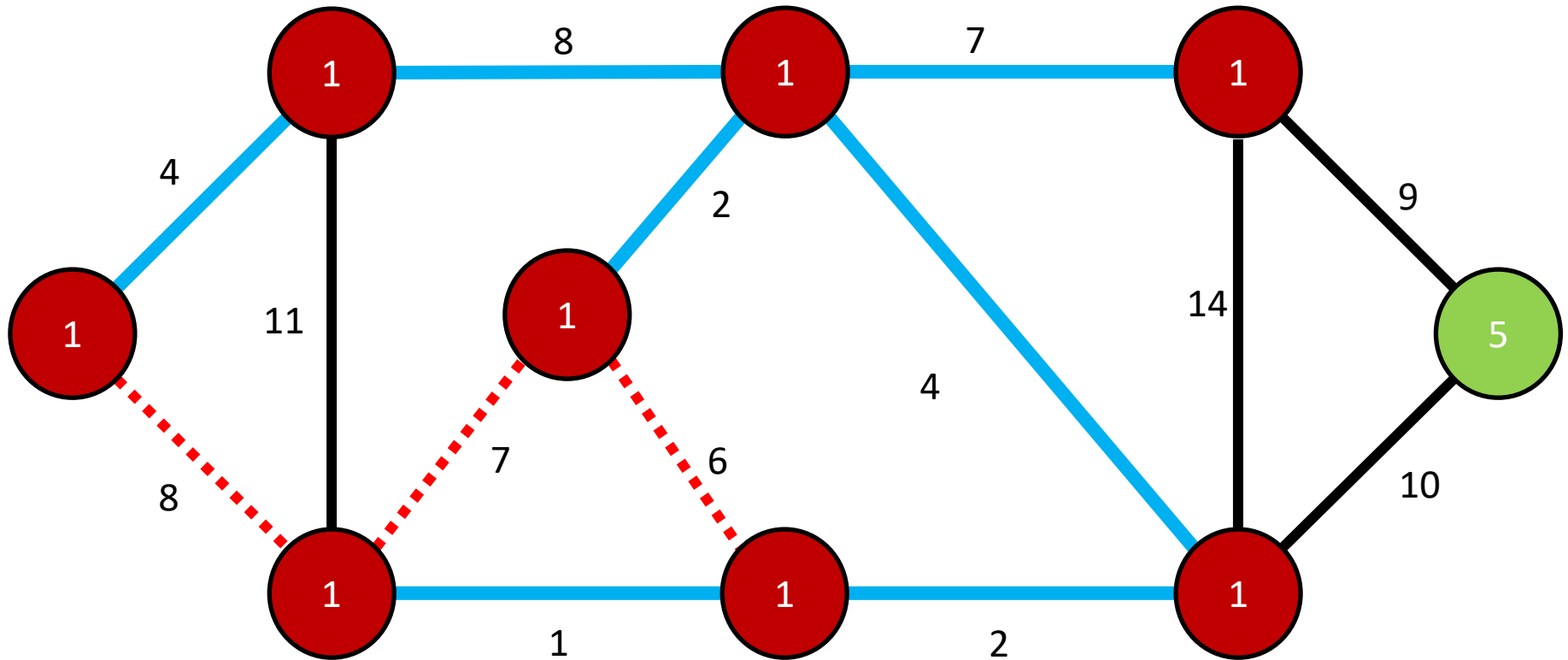
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



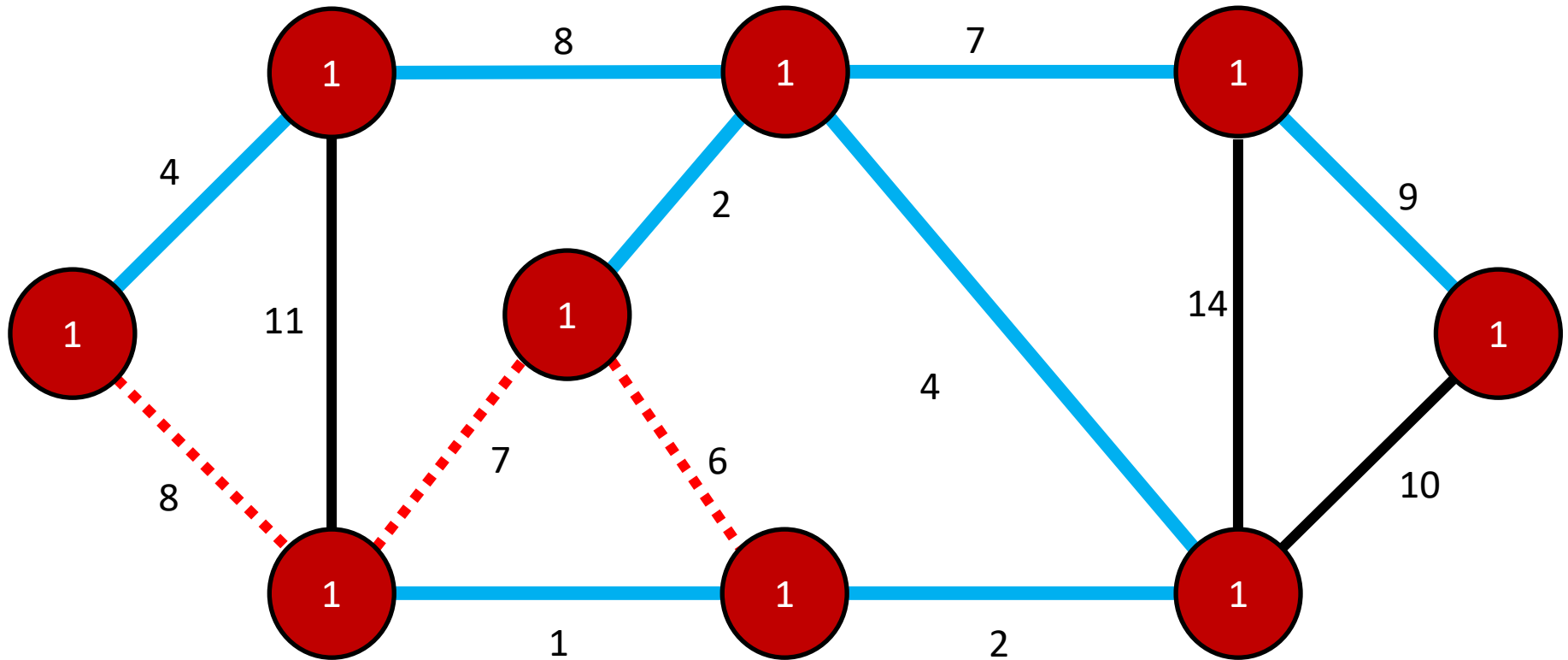
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



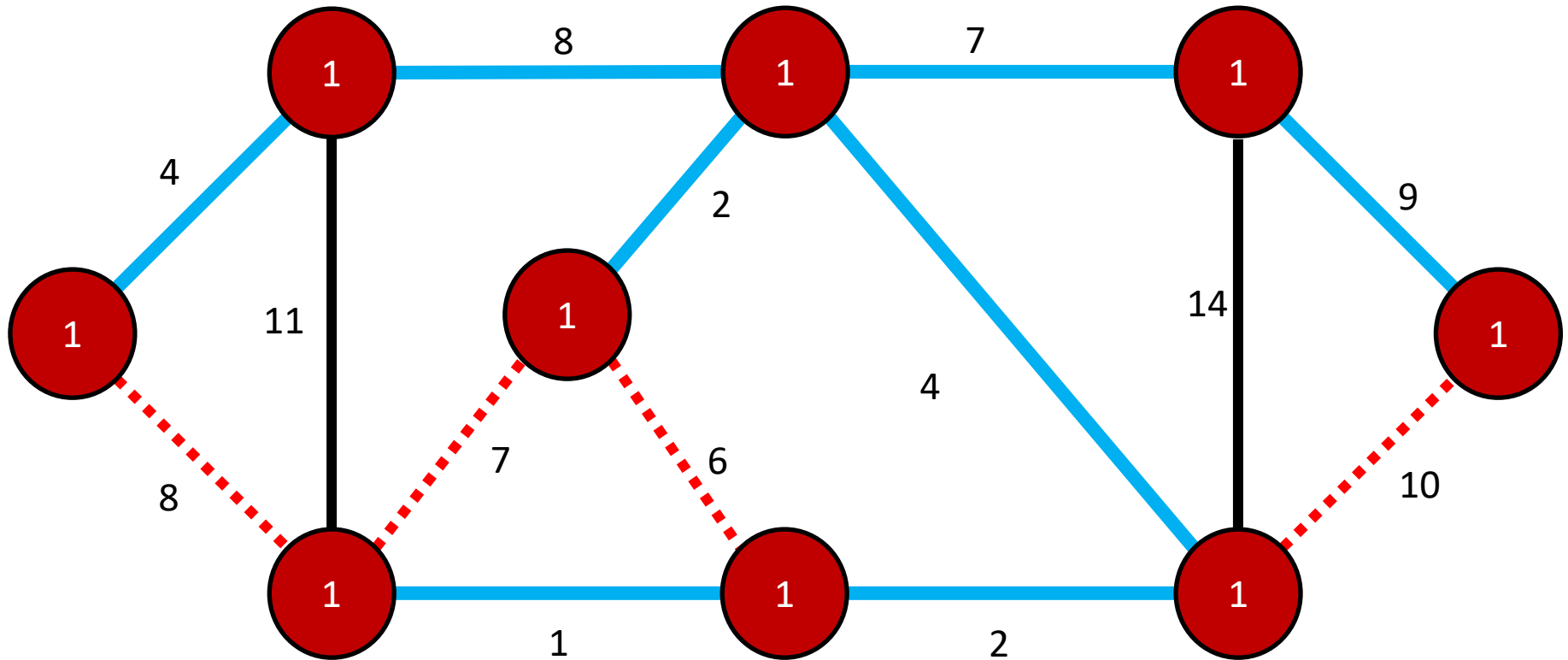
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



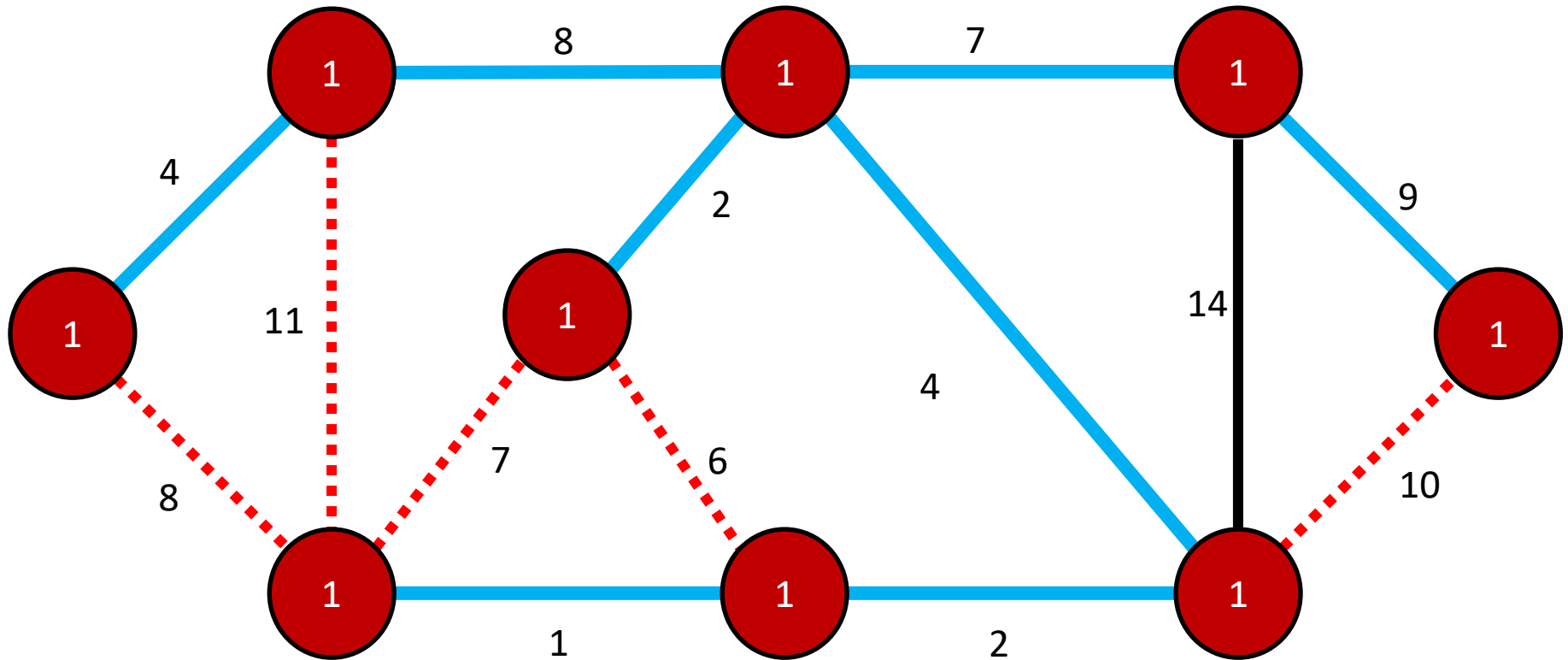
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



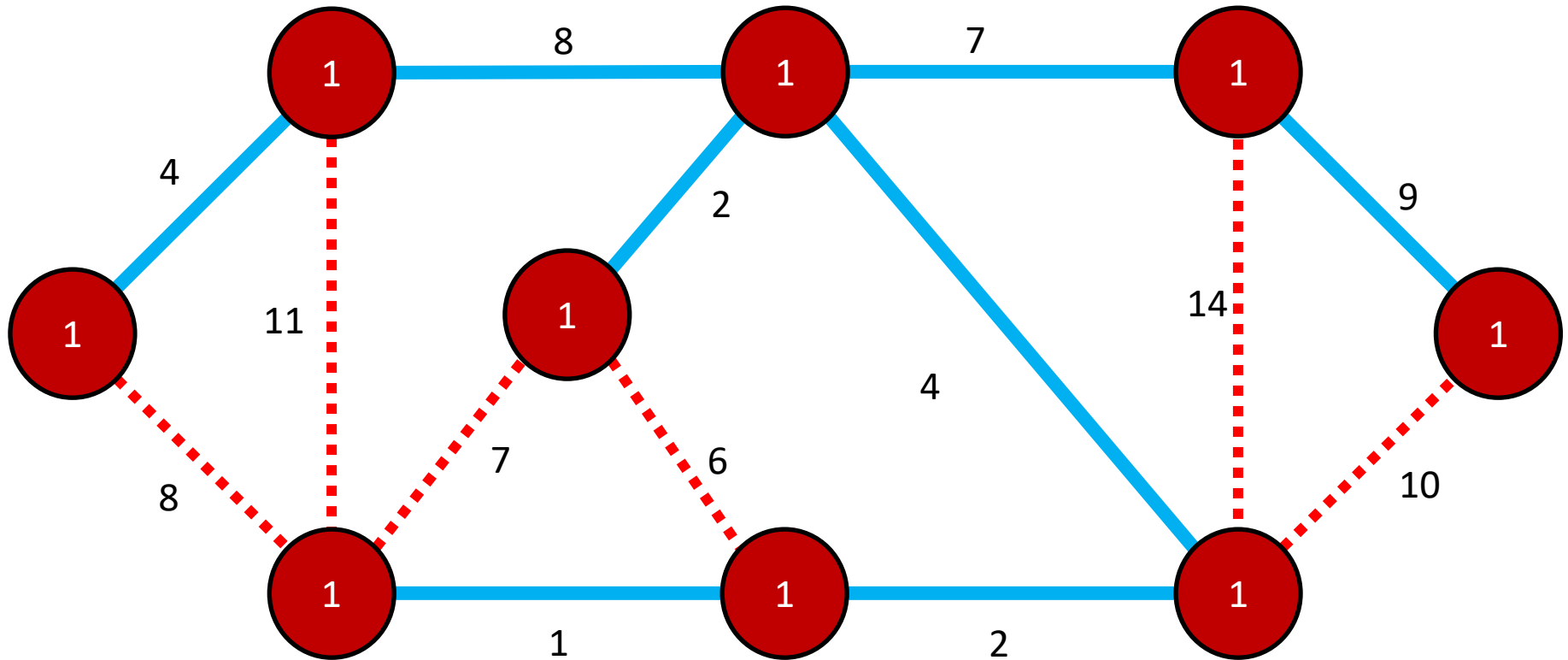
Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



Algoritmo de Kruskal

Evolução do algoritmo: À medida que as arestas são escolhidas, realizamos a fusão das árvores.



Algoritmo de Kruskal

Como implementar a verificação de ciclos?

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset; \quad \text{-- inicialização de } T$

para $j = 1, \dots, n$ faça

$S_j \leftarrow \{j\}; \quad \text{-- inicialização dos } n \text{ conjuntos disjuntos de vértices das } n \text{ árvores triviais}$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$, $j = 1, \dots, m-1$

para $j = 1, \dots, m$ faça

sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

se $p \neq q$ então

 acrescente e_j a $E(T)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset; \quad \text{-- inicialização de } T$

para $j = 1, \dots, n$ faça

$S_j \leftarrow \{j\}; \quad \text{-- inicialização dos } n \text{ conjuntos disjuntos de vértices das } n \text{ árvores triviais}$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$, $j = 1, \dots, m-1$

para $j = 1, \dots, m$ faça

sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

se $p \neq q$ então

$\text{acrescente } e_j \text{ a } E(T)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça

$S_j \leftarrow \{j\};$ -- *inicialização dos n conjuntos disjuntos de vértices das n árvores triviais*

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$, $j = 1, \dots, m-1$

para $j = 1, \dots, m$ faça

sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

se $p \neq q$ então

acrescente e_j a $E(T)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça
 $S_j \leftarrow \{j\};$ $O(n)$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$, $j = 1, \dots, m-1$

para $j = 1, \dots, m$ faça

sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

se $p \neq q$ então

 acrescente e_j a $E(T)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça
 $S_j \leftarrow \{j\};$ $O(n)$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$ $O(m \log m)$

para $j = 1, \dots, m$ faça

sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

se $p \neq q$ então

 acrescente e_j a $E(T)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça
 $S_j \leftarrow \{j\};$ $O(n)$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$ $O(m \log m)$

para $j = 1, \dots, m$ faça

 sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

 se $p \neq q$ então

 acrescente e_j a $E(T)$ $O(1)$

$S_p \leftarrow S_p \cup S_q$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça
 $S_j \leftarrow \{j\};$ $O(n)$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$ $O(m \log m)$

para $j = 1, \dots, m$ faça

 sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

 se $p \neq q$ então

 acrescente e_j a $E(T)$ $O(1)$

$S_p \leftarrow S_p \cup S_q$

Total do “para”: $O(m + n \log n)$

fim-se

fim-para

retorne T

Algoritmo de Kruskal

Cálculo da Complexidade

Algoritmo de Kruskal – nova versão

Entrada: grafo G com $V(G) = \{1, 2, \dots, n\}$

Saída: árvore geradora T de G com custo mínimo

$V(T) \leftarrow V(G); E(T) \leftarrow \emptyset;$ $O(n)$

para $j = 1, \dots, n$ faça
 $S_j \leftarrow \{j\};$ $O(n)$

seja $e_1, e_2, \dots, e_m$ uma ordenação das arestas de G onde $\text{peso}(e_j) \leq \text{peso}(e_{j+1})$ $O(m \log m)$

para $j = 1, \dots, m$ faça

 sejam S_p e S_q os conjuntos associados aos extremos da aresta e_j (onde $p \leq q$)

 se $p \neq q$ então

 acrescente e_j a $E(T)$ $O(1)$

$S_p \leftarrow S_p \cup S_q$

Total do “para”: $O(m + n \log n)$

fim-se

fim-para
retorne T

Total do algoritmo: $O(m \log m) = O(m \log n)$